UAX

TRABAJO FIN DE MÁSTER

MUIA
24-25

UNIVERSIDAD ALFONSO X EL SABIO

FACULTAD DE CIENCIAS DE LA EDUCACIÓN

Máster Universitario en Inteligencia Artificial



TRABAJO DE FIN DE MÁSTER

Modelos clásicos y cuánticos para la predicción de estabilidad proteica: un enfoque comparativo

Marco Antonio Roca Rodríguez
Óscar Fernández Mora

SEPTIEMBRE 2025

RESUMEN

La predicción del cambio en la energía libre de Gibbs (ΔΔG) asociado a mutaciones puntuales en proteínas constituye un desafío fundamental en biotecnología, con aplicaciones directas en el diseño de fármacos, la ingeniería de enzimas y la comprensión de enfermedades genéticas. Ante la complejidad de este problema, en este trabajo se han explorado tanto enfoques clásicos de aprendizaje automático como aproximaciones emergentes basadas en computación cuántica, con el objetivo de evaluar sus capacidades y limitaciones en un marco comparativo.

El estudio se llevó a cabo sobre el conjunto de datos S2648, ampliamente utilizado en la literatura para tareas de predicción de estabilidad proteica. Se diseñaron tres modelos principales: un regresor basado en XGBoost, una red neuronal totalmente conectada (FCNN) y una red neuronal cuántica híbrida (QNN) implementada con Qiskit. Para todos los casos, se aplicaron protocolos de preprocesamiento que incluyeron extracción de propiedades fisicoquímicas, información de vecinos en la secuencia y codificación one-hot de los residuos, además de reducción de dimensionalidad mediante PCA. La evaluación se realizó utilizando validación cruzada 5-fold y un conjunto de prueba fijo (80/20), aplicando métricas complementarias como PCC, MSE y MAE.

Los resultados muestran que XGBoost alcanzó el mejor rendimiento global, con un PCC de 0.703 y los errores más bajos (MSE=1.070, MAE=0.783) en el conjunto de prueba fijo, confirmando su solidez y estabilidad frente a distintas particiones. La FCNN obtuvo un desempeño intermedio (PCC=0.596, MSE=1.387, MAE=0.895), logrando capturar patrones no lineales, pero con mayor sensibilidad a la inicialización. Por su parte, la QNN presentó métricas más modestas en correlación (PCC≈0.54), aunque superó a la FCNN en error absoluto y cuadrático dentro del test fijo, evidenciando que los enfoques híbridos cuántico-clásicos ya pueden ofrecer resultados comparables a los de redes neuronales tradicionales, pese a las limitaciones actuales de hardware y software.

En conclusión, este trabajo demuestra la viabilidad de integrar técnicas clásicas y cuánticas en la predicción de estabilidad proteica, resaltando el potencial futuro de las redes cuánticas en biotecnología y abriendo una vía de exploración hacia modelos más precisos y eficientes conforme maduren las tecnologías cuánticas.

Palabras clave

Biotecnología, Inteligencia Artificial, Computación Cuántica, Proteínas, Estabilidad Proteica

ÍNDICE

Capítulo 1: Introducción 7

1. Motivación del proyecto 8
2. Presentación del proyecto y sus contenidos 9

Capítulo 2: Objetivos del TFM 11

1. Objetivo general 11
2. Objetivos específicos 11

Capítulo 3: Marco Teórico y estado del arte 13

1. Introducción 13
2. Inteligencia Artificial 14
   - 2.1. Definición y evolución de la IA 14
   - 2.2. Principales enfoques de la IA 15
   - 2.3. Aplicaciones actuales de la IA 19
3. Redes Neuronales 20
   - 3.1. Inspiración biológica 20
   - 3.2. Concepto y fundamentos matemáticos 21
   - 3.3. Primeros modelos y perceptrón 23
   - 3.4. Optimización y algoritmos de optimización de redes neuronales 26
   - 3.5. Técnicas de regularización 28
   - 3.6. Funciones de activación 29
4. Deep Learning y avances recientes 30
   - 4.1. Redes neuronales recurrentes (RNNs, LSTMs) 30
5. Consumo energético y sostenibilidad 31
6. Computación cuántica 32
   - 6.1. Qubits y operaciones básicas 34
   - 6.2. Operaciones básicas: Puertas lógicas y circuitos cuánticos 36
   - 6.3. Ventajas teóricas de la computación cuántica 37
   - 6.4. Limitaciones prácticas 38
7. Quantum Machine Learning (QML) y aplicaciones 38
   - 7.1. Redes neuronales cuánticas (QNNs): Concepto y teoría 39
   - 7.2. Diferencias clave con redes neuronales clásicas 40
   - 7.3. Arquitecturas principales de QNNs 41
8. Estado actual y desafíos de las QNNs 42
   - 8.1. Frameworks y herramientas prácticas 43
   - 8.2. Desafíos técnicos 43
   - 8.3. Futuras direcciones 44
9. Biotecnología. Inteligencia artificial y computación cuántica en este campo. 44

9.1. Biotecnología 45
9.2. Inteligencia artificial en la biotecnología 45
9.3. Computación cuántica en la biotecnología 46
9.4. Avances recientes, estado actual y futuro 47

Capítulo 4: Marco Metodológico 49

1. Población y muestra 49
2. Software utilizado 51
   2.1. Google Colab 51
   2.2. Visual Studio Code 52
   2.3. TensorFlow 52
   2.4. Scikit-learn 52
   2.5. Qiskit 53
   2.6. Qiskit Machine learning 54
   2.7. Otras herramientas 55
3. Hardware utilizado 56
   3.1. Hardware disponible en Google Colab 56
   3.2. Hardware Personal 57
   3.3. Hardware cuántico 58
4. Redes implementadas 58
   4.1. Planteamiento, proceso y metodología de desarrollo del proyecto 59
   4.2. Métricas de evaluación 61
   4.3. Preprocesamiento de datos 62
   4.4. Modelo XGBoost (SAAFEC-CEQ) 63
   4.5. Modelo FCNN 66
   4.6. Modelo cuántico análogo a XGBoost 70

Capítulo 5: Presentación de análisis, resultados y discusión 76

1. Análisis descriptivo de la muestra 76
2. Estadísticos y protocolos de evaluación 79
3. Resultados y discusión 80
4. Comparación con otros modelos de la literatura 88
5. Limitaciones 89

Capítulo 6: Conclusiones 90

1. Resumen del proyecto y conclusiones 90
2. Prospectiva 91
3. Consideraciones finales 92

Bibliografía 94

Capítulo 1: Introducción

En los últimos años, la Inteligencia Artificial (IA) ha experimentado un crecimiento exponencial a un ritmo vertiginoso, influenciado en gran parte por el aumento de la capacidad computacional, la disponibilidad de grandes conjuntos de datos y la creación de nuevos algoritmos de IA. Este avance tecnológico ha transformado industrias enteras, desde los campos de salud y finanzas hasta el entretenimiento y la manufacturación, redefiniendo tanto su funcionamiento como su estructura y hasta la forma en la que estas generan ingresos. Debido a esto, muchas empresas se han visto obligadas a cambiar su enfoque y centrarse en la IA y en la cultura del dato para no quedarse atrás.

La IA hasta hace unos años se percibía como algo futurista, pero actualmente es una herramienta esencial. Esta tecnología tiene una infinidad de utilidades que van desde el diagnóstico de enfermedades con una precisión comparable a la de los médicos hasta la conducción automática, e incluso la generación de contenido con una facilidad extraordinaria. Esto ha generado el interés de empresas de renombre mundial como Google, Nvidia, Amazon, etc. en la inversión del desarrollo, investigación y optimización de estas tecnologías, impulsando aún más su rápido crecimiento.

Aunque esta tecnología es relativamente reciente y su desarrollo se ha disparado en los últimos años, ya muestra claras limitaciones, como pueden ser el consumo energético y el coste computacional, entre otras. Estas limitaciones se vuelven especialmente críticas cuando se abordan problemas de alta complejidad, como los relacionados con la predicción de estructuras y funciones de proteínas —cuyo problema trabajaremos en este trabajo de fin de máster— que requieren grandes cantidades de procesamiento y modelos altamente precisos.

1. Motivación del proyecto

Para abordar estas limitaciones que se mencionan en el apartado anterior, se han explorado diversas estrategias, como la optimización del hardware actual, el diseño de nuevas arquitecturas algorítmicas y la implementación de enfoques híbridos. Sin embargo, los sistemas clásicos —tanto a nivel de hardware como de software— siguen suponiendo una barrera significativa. Ante este contexto, la computación cuántica ha emergido como una solución potencialmente revolucionaria, no solo por sus avances en hardware, sino también por el desarrollo de algoritmos cuánticos y modelos híbridos que aprovechan las propiedades de la mecánica cuántica.

La computación cuántica, fundamentada en los principios de la mecánica cuántica, promete capacidades de procesamiento superiores a las de los ordenadores clásicos al aprovechar fenómenos como la superposición y el entrelazamiento cuántico. Estas propiedades cuánticas permiten realizar cálculos de forma paralela, reduciendo de manera exponencial el tiempo necesario para resolver problemas de alta complejidad, que, con otros métodos tradicionales, requerirían recursos computacionales que a día de hoy se considerarían poco realistas.

Una de las áreas donde este potencial es especialmente prometedor es en la bioinformática y la biología computacional, particularmente en el estudio de proteínas. La predicción de la estructura tridimensional de proteínas, la simulación de interacciones moleculares o el diseño de nuevas cadenas proteicas, son tareas que pueden beneficiarse enormemente de la sinergia entre inteligencia artificial y computación cuántica.

Esta gigantesca capacidad computacional puede suponer la solución a algunas de las principales limitaciones actuales, sobre todo en el caso de las redes neuronales clásicas, permitiendo reducir el tiempo de entrenamiento y mejorando la capacidad de generalización al explorar de forma más amplia el espacio de soluciones. Gracias a esta nueva tecnología emergente, ha surgido una nueva arquitectura de red neuronal, las Redes Neuronales Cuánticas (Quantum Neural Networks o QNNs). Estas QNNs integran principios

cuánticos en aprendizaje automático, ofreciendo una solución escalable que podría superar las limitaciones de las redes clásicas.

2. Presentación del proyecto y sus contenidos

En este proyecto se aborda la evaluación del rendimiento de distintos modelos de regresión aplicados a un problema biomolecular concreto relacionado con proteínas. El estudio se centra en comparar una red neuronal clásica, un modelo inspirado en el algoritmo XGBoost (como el utilizado por SAAFEC-CEQ, el cual utilizaremos como base para la comparación de los modelos), y una versión cuántica análoga basada en técnicas de aprendizaje automático cuántico. El objetivo es analizar en qué medida la computación cuántica puede ofrecer mejoras frente a los enfoques clásicos, especialmente en contextos donde la complejidad del problema y las limitaciones computacionales son determinantes. Con ello, se busca aportar una visión teórico-práctica sobre el impacto real del aprendizaje cuántico en tareas de predicción biomolecular y cómo podría complementar o mejorar los modelos tradicionales.

El proyecto está estructurado en seis capítulos:

Capítulo 1, en este capítulo se introduce el contexto general y la motivación que impulsa el trabajo. También se presentan los contenidos para facilitar la lectura e introducir un poco más al lector al tema que se va a tratar.

En el Capítulo 2, veremos los objetivos que se buscan cumplir en este trabajo, tanto de manera general, como de manera específica, entrando en detalle a las distintas implementaciones y análisis a desarrollar.

En el Capítulo 3, se desarrolla el marco teórico, donde se explican los fundamentos de la inteligencia artificial, las redes neuronales clásicas, algunos modelos estadísticos y su posible adaptación cuántica. También se presentan los principios de la computación cuántica y sus

aplicaciones en el aprendizaje automático junto con una breve introducción a la biotecnología y algunos proyectos actuales desarrollados e.

El Capítulo 4, describe los objetivos a cumplir, la metodología del estudio, el planteamiento y desarrollo de la resolución de los problemas planteados, los conjuntos de datos utilizados, las herramientas empleadas y el enfoque que se ha llevado a cabo para comparar los distintos modelos junto con los problemas encontrados a lo largo de desarrollo.

En el Capítulo 5, se presentan y analizan los resultados obtenidos, comparando correlación de Pearson y errores cometidos entre otras. También se comentan las limitaciones y otros modelos de la literatura, comparando los modelos desarrollados con estos mismos.

Finalmente, el Capítulo 6, recoge las conclusiones del trabajo, las líneas futuras de investigación, una reflexión sobre el potencial de los modelos cuánticos en la resolución de problemas reales y todo lo aprendido durante el desarrollo de este trabajo.

Capítulo 2: Objetivos del TFM

En este capítulo, entraremos en detalle en los objetivos a resolver en este proyecto. Comenzaremos desde un punto de vista más general y nos iremos centrando en objetivos más concretos.

1. Objetivo general

El objetivo general de este TFM es evaluar y comparar el rendimiento de modelos clásicos y cuánticos en un problema de regresión relacionado con mutaciones de proteínas. Específicamente, se busca analizar la capacidad de distintos enfoques de aprendizaje automático para predecir la energía libre de cambio (ΔΔG) generada por mutaciones puntuales, utilizando el conjunto de datos s2648. En este trabajo se implementan y comparan una red neuronal clásica, un modelo estadístico clásico tipo XGBoost (inspirado en el enfoque del modelo SAAFEC-CEQ), y una versión cuántica análoga a este último, basada en principios de aprendizaje automático cuántico. El propósito es determinar si las técnicas cuánticas pueden ofrecer ventajas reales en cuanto a rendimiento, eficiencia y viabilidad computacional en este contexto biomolecular.

2. Objetivos específicos

A continuación, se exponen los objetivos específicos desarrollados en los que se entrará en detalle a lo largo de esta memoria:

1. Seleccionar, explorar y preparar un conjunto de datos representativo para el estudio de mutaciones en proteínas —en este caso, el *dataset* s2648— realizando su análisis exploratorio, limpieza, selección de características, normalización y transformación. Se llevará a cabo un

preprocesamiento que permita adaptar el conjunto de datos para su uso efectivo en distintos modelos, tanto clásicos como cuánticos.

2. Diseñar e implementar los modelos clásicos y cuánticos: una red neuronal tradicional con TensorFlow, un modelo de *boosting* clásico inspirado en SAAFEC-CEQ, y una versión híbrida clásico-cuántica análoga a XGBoost mediante *frameworks* como Qiskit Machine Learning.

3. Comparar el rendimiento de los modelos clásicos y cuánticos, evaluando métricas como correlación de Pearson y errores cuadráticos y medios.

4. Analizar los resultados obtenidos para identificar las ventajas, limitaciones y posibles escenarios de aplicación de los modelos cuánticos en biomedicina computacional, especialmente en contextos de predicción sobre mutaciones de proteínas.

Capítulo 3: Marco Teórico y estado del arte

Este estudio contribuirá a comprender el estado actual de la inteligencia artificial, los modelos clásicos de regresión como XGBoost entre otros, las redes neuronales y las redes neuronales cuánticas (QNNs), así como las posibles aplicaciones de estas tecnologías en la resolución de problemas relacionados con mutaciones de proteínas. Además, se abordarán los retos técnicos actuales que deben superarse para lograr una integración práctica de los modelos cuánticos en el análisis biomolecular, especialmente en tareas de predicción de la estabilidad proteica.

1. Introducción

Como se ha comentado anteriormente, la inteligencia artificial ha experimentado un crecimiento exponencial en los últimos años. Desde las primeras formulaciones teóricas hasta su implementación en modelos complejos como redes neuronales profundas o distintos algoritmos de aprendizaje automático, esta tecnología ha demostrado un gran potencial en múltiples ámbitos, incluyendo la biología computacional. En particular, herramientas como XGBoost se han convertido en referentes dentro del análisis de mutaciones en proteínas, como lo demuestra su uso en modelos como SAAFEC-CEQ.

Sin embargo, la computación cuántica ha abierto nuevas posibilidades en el aprendizaje automático, dando lugar a nuevas líneas de investigación como las redes neuronales cuánticas (QNNs) y los modelos híbridos, que podrían ofrecer ventajas computacionales en problemas de alta complejidad.

Este capítulo tiene como principal objetivo proporcionar una visión general del estado del arte de la inteligencia artificial aplicada a la predicción de estabilidad proteica, comenzando por su evolución hasta los modelos actuales como redes neuronales y algoritmos de *boosting*. Posteriormente, se explorarán los principios fundamentales de la computación cuántica, sus elementos básicos (como puertas cuánticas y qubits), y su integración en el

campo del aprendizaje automático mediante técnicas como los circuitos cuánticos y las QNNs. Finalmente, se presentarán las posibles ventajas de los modelos cuánticos frente a los clásicos, sus aplicaciones más prometedoras en el ámbito de las proteínas, así como los desafíos técnicos que aún deben superarse.

A continuación, se desarrollan estos contenidos con el fin de proporcionar un marco teórico sólido que sirva como base para el análisis comparativo posterior entre los modelos clásicos (XGBoost y redes neuronales) y sus versiones cuánticas aplicada al problema de regresión de estabilidad proteica.

2. Inteligencia Artificial

En este apartado hablaremos sobre qué es la IA, como ha evolucionado a lo largo de los años, los principales enfoques de la IA y algunas de las principales aplicaciones de esta. El propósito de este apartado es realizar una breve introducción a este campo para facilitar la comprensión de los siguientes apartados y entender cómo se ha llegado a ese punto en el desarrollo de esta tecnología.

2.1. Definición y evolución de la IA

La IA es un campo de la informática que se enfoca en crear sistemas que puedan realizar tareas que normalmente requieren inteligencia humana, como el aprendizaje, el razonamiento y la percepción. Aunque la IA moderna se basa en modelos de aprendizaje automático y redes neuronales, sus fundamentos teóricos datan de la década de 1950, cuando Alan Turing planteó la posibilidad de que las máquinas tuviesen la capacidad de pensar o razonar.

Inicialmente, la IA se basó en los sistemas expertos, los cuales eran sistemas de reglas explícitas que tomaban decisiones teniendo en cuenta una base de conocimientos previos. Sin embargo, dichas aproximaciones demostraron ser ineficientes a la hora de resolver problemas complejos. Tras un tiempo sin hacer avances significativos, llegó el aprendizaje automático y se comenzaron

a observar avances significativos. El aprendizaje automático les permitió a las máquinas la capacidad de aprender patrones a partir de un conjunto de datos sin necesidad de una programación explícita, lo que revolucionó por completo este campo. Este cambio ha sido crucial en campos como la medicina, la industria farmacéutica o la biología computacional, donde el volumen y complejidad de los datos dificultan un enfoque tradicional.

El aprendizaje automático se divide en distintas categorías como aprendizaje supervisado, no supervisado y aprendizaje por refuerzo, según el tipo de entrenamiento, en las cuales entraremos en más detalle en el próximo apartado, comentando las características de cada una de estas categorías.

A medida que la dificultad de los problemas aumentaba, las técnicas de aprendizaje automático mostraban limitaciones evidentes, sobre todo en tareas como reconocimiento de imágenes, procesamiento del lenguaje natural y la toma de decisiones en entornos dinámicos. Debido a esto y la necesidad de manejar datos más complejos y extraer representaciones más abstractas, el aprendizaje automático fue evolucionando hacia el aprendizaje profundo de manera natural.

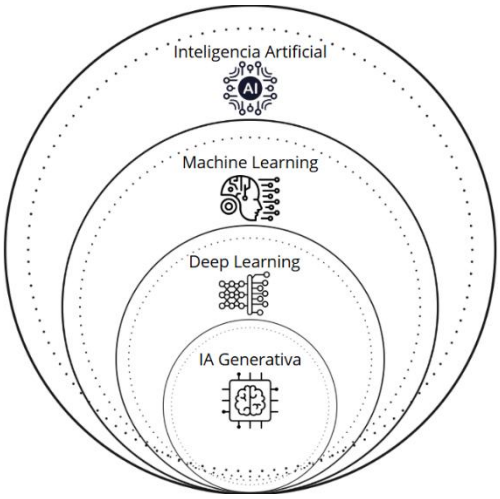
El aprendizaje profundo o *Deep Learning* (DL), destaca principalmente por el uso de redes neuronales artificiales profundas, las cuales han permitido realizar avances significativos en campos donde el aprendizaje automático mostraba limitaciones. Entre estos campos se incluyen no solo áreas clásicas como la visión por computadora o el procesamiento del lenguaje natural como se ha comentado anteriormente, sino también disciplinas emergentes como la biología computacional y la biotecnología.

2.2. Principales enfoques de la IA

A medida que la IA ha evolucionado, se han adoptado distintos enfoques que permiten desarrollar sistemas centrados en resolver problemas de forma automática. Cada uno de estos enfoques está pensado para un tipo de problema distinto, por lo que tiene sus propias características donde se incluyen las ventajas y limitaciones de estos. Estas ventajas y limitaciones

han supuesto que cada enfoque se dedique a ámbitos diferentes, desde la industria hasta la investigación científica. En los siguientes subapartados se van a detallar los principales enfoques dentro de la IA.

Figura 1. Campos de la inteligencia artificial



Nota: En esta imagen podemos apreciar cómo estos campos de estudio están anidados dentro de otros y como su especificación crece a medida que aumenta la profundidad.

5. IA Simbólica o basada en reglas

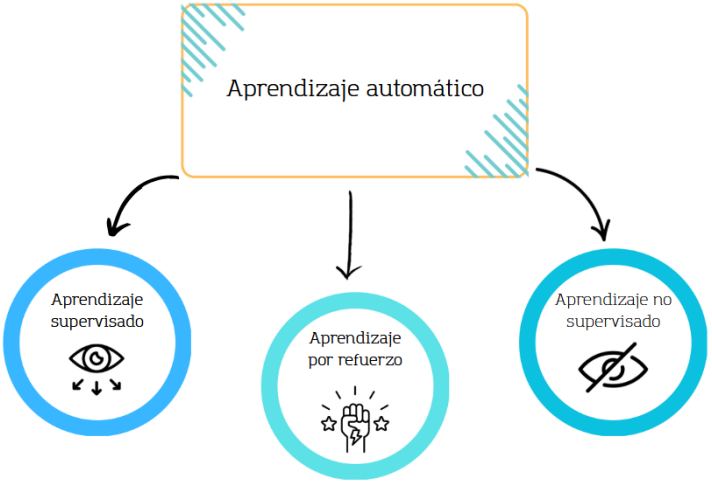
La inteligencia artificial simbólica (IA simbólica) es un enfoque de la inteligencia artificial que usa símbolos y reglas lógicas para procesar información y tomar decisiones basándose en conocimientos previos. El objetivo principal de este enfoque es la construcción de algoritmos inteligentes que puedan pensar y razonar como humanos.

Un ejemplo destacado de la IA simbólica son los sistemas expertos. Esos sistemas están enfocados en solucionar un problema concreto intentando simular el razonamiento humano como lo haría un profesional en la materia. Un ejemplo de esto es la utilización de estos sistemas en el área de diagnóstico médico, donde se usan antes de conocimientos para detectar enfermedades a partir de ciertos síntomas.

6. Aprendizaje automático o Machine Learning (ML)

El aprendizaje automático representa un gran cambio en comparación a la IA simbólica ya que, en lugar de depender de reglas y conocimientos previos, permite que el sistema aprenda a partir de los datos. Los modelos de ML usan algoritmos para identificar patrones y mejoran su rendimiento con la experiencia.

Figura 2. Tipos de aprendizaje automático



Nota: En esta imagen podemos observar como el aprendizaje automático se divide en 3 tipos en función del tipo de aprendizaje, valga la redundancia.

- **Aprendizaje supervisado:** los modelos son entrenados con datos etiquetados, donde la salida está asociada con una salida y se aprenden patrones a partir de las variables de entrada y salida.
- **Aprendizaje no supervisado:** los modelos buscan patrones en los datos sin ninguna referencia previa.
- **Aprendizaje por refuerzo:** se basa en la interacción de un agente dentro de un entorno dinámico donde éste aprende a maximizar una recompensa mediante la con el fin de optimizar su rendimiento.

7. Aprendizaje profundo o Deep Learning (DL)

El aprendizaje profundo es una rama del aprendizaje automático que se basa en el uso de algoritmos para modelar y comprender datos complejos. A diferencia de ML, donde los algoritmos dependen en gran medida de la ingeniería manual de características, *deep learning* automatiza este proceso mediante capas ocultas para transformar los datos de entrada en representaciones más significativas.

Cada capa en una red neuronal profunda transforma la información de entrada progresivamente lo que facilita la detección de patrones complejos y la toma de decisiones en datos no estructurados. Esto hace que el DL sea especialmente útil en la visión por computadora, el procesamiento del lenguaje natural y la robótica.

Dentro del DL, existen diferentes tipos de arquitecturas diseñadas para abordar problemas concretos:

- **Redes Neuronales Convolucionales (CNNs):** útiles para la visión por computadora.
- **Redes Neuronales Recurrentes (RNNs) y Transformers:** útiles en el procesamiento del lenguaje natural y la generación de texto.
- **Autoencoders y Redes Adversarias Generativas (GANs):** útiles en la generación de datos sintéticos, compresión de información y detección de anomalías.

8. Modelos generativos en IA

Uno de los avances más importantes en la IA ha sido el desarrollo de modelos generativos, capaces de crear nuevo contenido a partir de un conjunto de datos. Entre las arquitecturas más destacadas podemos encontrar:

- **Redes Adversarias Generativas (GANs):** Las GANs son un tipo de modelo de inteligencia artificial que genera datos realistas a partir de un conjunto de datos de entrenamiento.
- **Modelos de difusión:** Los modelos de difusión son modelos generativos que se utilizan principalmente para la generación de imágenes y otras tareas de visión artificial. Las redes neuronales basadas en difusión se entrenan mediante *deep learning* para "difundir" progresivamente muestras con ruido aleatorio y, a continuación, invertir ese proceso de difusión para generar imágenes de alta calidad.

2.3. Aplicaciones actuales de la IA

Como hemos visto hasta este momento, la inteligencia artificial tiene un gran potencial para transformar sectores enteros, desde la medicina y la investigación científica hasta la industria y los servicios. En los últimos años, también ha comenzado a ocupar un papel clave en áreas como la biología computacional y la bioinformática, especialmente en el estudio de proteínas, donde ha demostrado un rendimiento sobresaliente en tareas de predicción estructural y análisis funcional. Entre sus aplicaciones más comunes actualmente podemos encontrar algunas de las vistas a lo largo de este apartado como:

- **Visión por computadora** (reconocimiento facial, diagnóstico médico basado en imágenes, detección de objetos en tiempo real).
- **NLP** (traducción automática, generación de texto y asistentes virtuales)
- **Robótica e IA aplicada** (vehículos autónomos, robots industriales y sistemas de asistencia médica)
- **Optimización y predicción** (IA en mercados financieros, mantenimiento predictivo en la industria y gestión de energía)

También podemos destacar el uso de la IA en el campo de la biología computacional. Esta herramienta está ganando una gran importancia gracias a la cantidad de avances importantes que se han conseguido con la ayuda de diversos algoritmos y modelos. Estos modelos de aprendizaje automático y

profundo permiten predecir la estabilidad de proteínas ante mutaciones, estimar cambios en la energía libre (ΔΔG), y generar estructuras tridimensionales precisas a partir de secuencias, como en el caso de AlphaFold, uno de los modelos actuales más potentes diseñados para este problema. También se utilizan para diseñar nuevas variantes funcionales o detectar mutaciones patológicas con mayor precisión.

En la actualidad, la investigación en IA se centra en mejorar la interpretabilidad de los modelos, reducir su consumo energético y desarrollar sistemas más robustos, éticos y sostenibles. Además, la posible combinación de la IA con la computación cuántica abre nuevas oportunidades para resolver problemas altamente complejos —como los asociados al diseño de proteínas o a la simulación molecular— de una manera más eficiente y escalable.

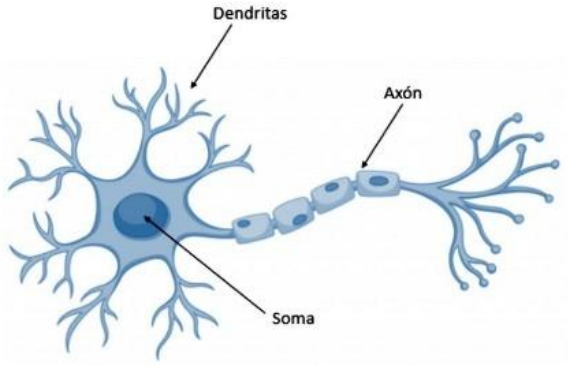
3. Redes Neuronales

Las redes neuronales artificiales son algoritmos que imitan el funcionamiento del cerebro humano. Su función es procesar información mediante neuronas artificiales agrupadas en capas. Desde su creación, estas herramientas han demostrado una gran capacidad en la resolución de diversos problemas complejos. Tanto el interés en la resolución de problemas complejos como los grandes volúmenes de datos y el avance del hardware ha tenido como resultado que las redes neuronales artificiales evolucionen a un gran ritmo.

3.1. Inspiración biológica

Las redes neuronales artificiales están inspiradas en el funcionamiento del sistema nervioso humano, específicamente en cómo las neuronas procesan la información. En el cerebro, las neuronas reciben señales a través de las dendritas, procesan la información y transmiten impulsos eléctricos mediante los axones, formando redes interconectadas capaces de aprender y adaptarse. En la *Figura 3* podemos apreciar una neurona con los equivalentes de las redes neuronales artificiales. Las dendritas que cumplen la función de capa de entrada, el soma como función de activación y el axión como capa de salida.

Figura 3. Neurona biológica



Nota: en esta imagen podemos observar una neurona, con sus dendritas, equivalentes a entradas, el soma, equivalente a una función de activación y el axón, equivalente a la salida.

De forma análoga, las redes neuronales artificiales constan de neuronas artificiales agrupadas en capas. Estas neuronas reciben múltiples entradas y generan un resultado que transmiten a la siguiente capa, donde se repite el proceso.

A diferencia del cerebro humano, que aprende de manera flexible y con poca información, estas redes neuronales requieren grandes cantidades de datos y un entrenamiento intensivo para lograr un rendimiento óptimo. Actualmente, con los avances en el desarrollo hardware y diversos algoritmos, la capacidad de aprendizaje de estas herramientas cada vez se acerca más a ciertos aspectos de la inteligencia biológica.

3.2. Concepto y fundamentos matemáticos

Las redes neuronales artificiales tienen como objetivo principal el aprendizaje de patrones y relaciones de datos a partir de ejemplos dados. Estas redes se estructuran en capas de neuronas que reciben, procesan y transmiten información. Dichas capas se dividen en 3 tipos por norma general:

- **Capa de entrada:** es la capa que recibe los datos iniciales.
- **Capas ocultas:** son las capas encargadas de procesar la información.
- **Capa de salida:** es la capa que se encarga de generar la respuesta del sistema.

Dentro de los fundamentos matemáticos, podemos comenzar por las neuronas, donde cada neurona de una capa se encarga de realizar una transformación matemática sobre los datos que recibe. La ecuación que define el comportamiento de las neuronas es la siguiente:

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

Donde:

- xi representa la entrada de la neurona
- wi representa el peso asociado a cada entrada
- b es el sesgo que permite ajustar la salida del modelo
- f es la función de activación

Dentro de las funciones de activación, algunas de las más comunes y utilizadas son:

- **Sigmoide:** útil para problemas de clasificación binaria.
- **ReLU (Rectified Linear Unit):** mejora la eficiencia de redes profundas
- **Softmax:** utilizada en tareas de clasificación multiclase

Más adelante, entraremos en detalle en diversas funciones de activación.

El aprendizaje de una red neuronal se basa en la minimización de una función de pérdida, utilizando algoritmos de optimización como el de descenso del gradiente estocástico o Adam, los cuales son unos de los más utilizados. Estos

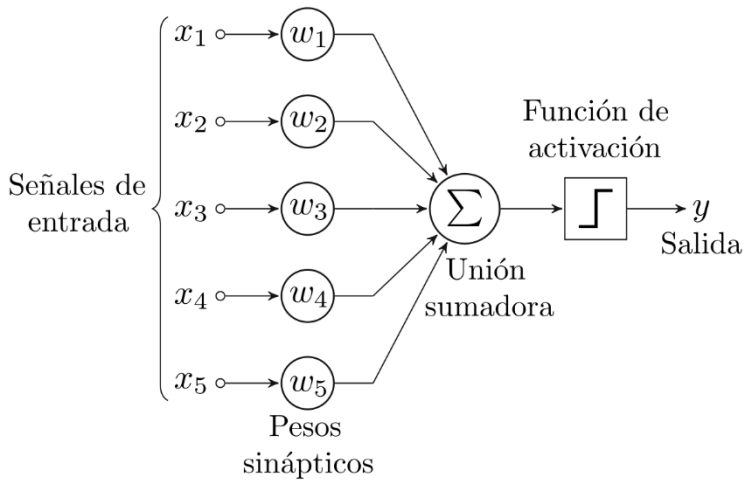
algoritmos se encargan de ajustar los pesos de la red a través del cálculo del gradiente mediante la retro propagación.

3.3. Primeros modelos y perceptrón

El perceptrón fue uno de los primeros modelos de redes neuronales artificiales, desarrollado por Frank Rosenblatt en 1958 con la intención de clasificar patrones. Está inspirado en el cerebro humano y se basa en la combinación de entradas ponderadas, un umbral y una función de activación binaria.

El perceptrón de Rosenblatt está compuesto por neuronas de entrada, pesos sinápticos, una función de activación y una salida binaria. El funcionamiento de este aprendizaje se basa en la actualización de los pesos con un algoritmo supervisado. Podemos observar su estructura en la *Figura 4*.

Figura 4. Perceptrón



Nota: en esta imagen podemos observar un perceptrón con sus distintos elementos.

A pesar de su éxito inicial, más adelante se demostró que el perceptrón no era capaz de resolver problemas no lineales. Esta limitación fue la causa por la que se comenzaron a desarrollar las redes neuronales multicapa y el

algoritmo de retro propagación del error, factores que fueron clave posteriormente en el campo de aprendizaje profundo.

Las redes neuronales multicapa funcionan principalmente gracias a la propagación hacia delante (*forward propagation*). Esto es el movimiento de los datos desde la entrada a la salida a través de capas sucesivas, procesando los datos mediante la aplicación de pesos y funciones de activación. A las redes neuronales que funcionan de esta manera se les llama red neuronal prealimentada (*feedforward neural network*).

Otra parte clave de estas redes es el algoritmo de retro propagación (*back propagation*), el cual es un algoritmo utilizado en aprendizaje supervisado que consiste en calcular el error de salida y propagarlo hacia las capas anteriores para ajustar los pesos mediante el descenso del gradiente y la regla de la cadena. Este algoritmo busca la minimización del error entre el resultado obtenido y el resultado esperado. Este algoritmo es uno de los principios más importantes del *deep learning*, ya que permite entrenar modelos de redes neuronales profundas de forma eficiente. Sin este algoritmo la optimización de las redes multicapa sería extremadamente difícil, dado que los errores no se propagarían de manera efectiva.

Una vez se obtiene un resultado, este se compara con la salida esperada mediante una función de costo, la cual mide el error cometido por el modelo. Hay distintas funciones comúnmente utilizadas para poder medir el error cometido por el modelo. Algunas de estas funciones son:

- **Error cuadrático medio** (MSE, *Medium Square Error*): esta función está definida por la ecuación:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

  Es una función que se utiliza en problemas de regresión. Penaliza los errores grandes, lo que lo hace sensible a valores atípicos.

- **Error medio absoluto** (MAE, *Mean Absolute Error*): esta función está definida por la ecuación:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Se emplea en regresión, a diferencia del MSE, trata los errores de manera más uniforme ya que no los eleva al cuadrado y los hace menos sensibles a valores atípicos.

- **Entropía cruzada (*Cross-Entropy Loss*):** Se utiliza para los problemas de clasificación tanto binaria cómo clasificación multiclase. Es ideal cuando las salidas en formato de probabilidad, por ejemplo, cuando se utiliza la función de activación *softmax*. Como se ha comentado, dependiendo del tipo de problema, tenemos las siguientes funciones:

  - **Binaria:**

$$L = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log \hat{y}_i (1 - y_i) \log(1 - \hat{y}_i)]$$

  - **Multiclase**

$$L = -\frac{1}{n}\sum_{i=1}^{n}\sum_{i=1}^{c} y_{ic} \log(\hat{y}_{ic})$$

En modelos predictivos aplicados a la bioinformática, las funciones de pérdida como el MSE o MAE son fundamentales para medir la desviación entre la predicción y el valor experimental, permitiendo entrenar modelos que optimicen estas predicciones.

3.4. Optimización y algoritmos de optimización de redes neuronales

Para minimizar la función de costo y mejorar el rendimiento del modelo, se utilizan algoritmos de optimización, los cuales ajustan de forma iterativa los pesos y sesgos de esta para minimizar la función de costo. Los algoritmos de optimización son herramientas matemáticas y computacionales diseñadas para encontrar la mejor solución posible a un problema, dentro de un conjunto definido de posibilidades. Estos algoritmos consideran una función objetivo que se desea maximizar o minimizar, junto con un conjunto de restricciones. Esto resulta especialmente relevante en contextos como la biotecnología, donde modelos de predicción de proteínas deben ajustarse finamente a los datos experimentales obtenidos en condiciones biológicas complejas. Algunos de los optimizadores más utilizados son:

- **Descenso del gradiente (GD):** el descenso de gradiente es un algoritmo de optimización que se utiliza habitualmente para entrenar modelos de ML y redes neuronales. Este algoritmo entrena modelos de ML mediante la minimización de errores entre los resultados previstos y los reales. Se basa en la actualización iterativa de los pesos mediante la ecuación:

$$W = W - \eta \cdot \nabla J(W)$$

Donde:

- W representa los pesos del modelo.
- η (*learning rate*) es la tasa de aprendizaje, es el parámetro que controla el tamaño de los pasos en cada iteración.
- ∇J(W) es el gradiente de la función de costo con respecto a los pesos.

Un *learning rate* demasiado grande puede hacer que el modelo no converja o que oscile sin llegar a la solución óptima deseada, mientras que un *learning rate* demasiado bajo sea demasiado lento y costoso computacionalmente.

Otros parámetros que pueden influir son el número de iteraciones (épocas) y el tamaño del subconjunto de datos para la actualización de pesos en cada iteración.

- **Descenso del gradiente estocástico (SGD):** el descenso del gradiente estocástico es una variante del descenso del gradiente que se centra en actualizar los pesos usando una muestra aleatoria del conjunto de datos en lugar de utilizar todo el conjunto. La ecuación de actualización de los pesos es muy similar a la de GD:

$$W = W - \eta \cdot \nabla J(W, x_i)$$

Donde:

- W representa los pesos del modelo.
- η (*learning rate*) es la tasa de aprendizaje, es el parámetro que controla el tamaño de los pasos en cada iteración.
- ∇J(W) es el gradiente de la función de costo con respecto a los pesos.
- x_i es una muestra individual del conjunto de datos.

SGD añade aleatoriedad al proceso de optimización, lo que ayuda a evitar mínimos locales. Hay diferentes variables que se utilizan para mejorar el rendimiento de este optimizador, como, por ejemplo:

- **Momentum**: Introduce una memoria del gradiente anterior para suavizar las actualizaciones y acelerar la convergencia.
- **Decay Learning rate**: Reduce progresivamente la tasa de aprendizaje para mejorar la estabilidad a medida que el modelo se acerca al óptimo.

- **Adam (Adaptive Moment Estimation):** Adam es una extensión del SGD que combina el momentum con una estimación adaptativa de la tasa de aprendizaje para cada parámetro. Se define por las siguientes ecuaciones:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla J(W)$$

$$v_t = \beta_2 v_t + (1-\beta_2)\left(\nabla J(W)\right)^2$$

$$\widehat{m_t} = \frac{m_t}{1-\beta_1^t}, \widehat{v_t} = \frac{v_t}{1-\beta_2^t}$$

$$W = W - \eta \frac{\widehat{m_t}}{\sqrt{\widehat{v_t} + \varepsilon}}$$

Donde:

- m_t y v_t son los promedios móviles del primer y segundo momento del gradiente.
- β_1 y β_2 son hiperparámetros que controlan la caída exponencial de los momentos.
- ε es un valor pequeño que evita divisiones por cero.

Adam es de los optimizadores más utilizados ya que combina las ventajas de SGD con Momentum y adaptabilidad en la tasa de aprendizaje, lo que lo hace una opción muy robusta y eficiente para distintos problemas de optimización.

3.5. Técnicas de regularización

Las técnicas de regularización son técnicas que ayudan a mejorar la capacidad de generalización del modelo y a prevenir el sobreajuste (*overfitting*). Algunas de las más utilizadas son:

- **Dropout**: se desactivan algunas neuronas en cada capa aleatoriamente evitando que el modelo dependa en exceso de determinadas neuronas mejorando su capacidad de generalización.
- **Batch Normalization**: normaliza las activaciones de cada capa durante el entrenamiento, acelerando la convergencia y mejorando la estabilidad del aprendizaje.

- **Regularización L1 y L2**: Se agregan términos de penalización a la función de costo para evitar que los pesos crezcan demasiado, reduciendo el riesgo de sobreajuste.

3.6. Funciones de activación

Las funciones de activación son un componente esencial en las redes neuronales, ya que introducen la no linealidad en el modelo, permitiendo así que el modelo pueda aprender relaciones complejas en los datos. Estas funciones de activación se encargan de transformar la entrada de una salida antes de enviarla a la siguiente capa, algo que tiene un impacto directo en la capacidad de aprendizaje y convergencia de la red.

Durante el desarrollo de las redes neuronales, se han propuesto diferentes funciones de activación, cada una de estas con sus ventajas y desventajas respecto a otras, dependiendo del tipo de problema a abordar. Algunas de las más utilizadas son las que se han mencionado anteriormente:

- **Sigmoide (σ):** Su salida está entre 0 y 1, es útil para clasificación binaria cómo probabilidad. Puede sufrir desvanecimiento del gradiente y no está centrada en 0, lo que puede dificultar el aprendizaje.
- **Tangente hiperbólica (Tanh):** su rango va de -1 a 1, está centrada en 0, lo que mejora la propagación del gradiente. También puede sufrir desvanecimiento del gradiente en valores extremos.
- **ReLU:** Devuelve 0 para valores negativos y x para positivos, su rango es $[0, \infty)$. Es eficiente, no sufre tanto el desvanecimiento del gradiente, pero puede producir neuronas muertas si recibe muchos valores negativos.

Las funciones de activación no solo afectan a la capacidad de la red neuronal para modelar relaciones, sino que también influye en la eficiencia del entrenamiento y la estabilidad del modelo. Según qué función de activación seleccionemos, podemos acelerar o ralentizar la convergencia, alterar o afectar la capacidad de generalización y la estabilidad del gradiente. En

tareas de predicción biomolecular, como las enfocadas en mutaciones de proteínas, la selección adecuada de la función de activación influye directamente en la precisión del modelo, especialmente cuando se trabaja con variables bioquímicas continuas o altamente no lineales.

4. Deep Learning y avances recientes

Como ya hemos comentado previamente, el DL ha revolucionado el aprendizaje automático al permitir que los modelos aprendan representaciones jerárquicas y abstractas de los datos. Gracias a avances en arquitecturas de redes, la optimización de estas y la potencia computacional, los modelos actuales abordan con facilidad tareas muy complejas que antes estos mismo no podían en problemas de visión por computadora, NLP y generación de contenido.

Las redes neuronales han evolucionado significativamente, desarrollando arquitecturas especializadas para el procesamiento de distintos datos. A diferencia de las MLP tradicionales, las cuales procesan la información en capas completamente conectadas, las arquitecturas más avanzadas de redes neuronales están diseñadas para mejorar el rendimiento y reducir la capacidad computacional.

En este capítulo, veremos las redes neuronales recurrentes, las cuales utilizaremos durante el desarrollo de este trabajo y cómo las redes neuronales han revolucionado el campo de la bioquímica y el trabajo con proteínas.

4.1. Redes neuronales recurrentes (RNNs, LSTMs)

El procesamiento de datos como texto, audio y series temporales requiere el uso de arquitecturas especializadas. Las redes neuronales recurrentes fueron una de las primeras soluciones a estos problemas ya que tienen la capacidad de almacenar datos en su “memoria”, puesto a que toman la información de las entradas anteriores para influir en la entrada y salida actuales.

A pesar de su utilidad, las RNNs tradicionales tienen problemas con el desvanecimiento del gradiente, lo que dificulta que estas puedan aprender dependencias a largo plazo. Como posible solución a esto se desarrollaron variantes de las RNNs como:

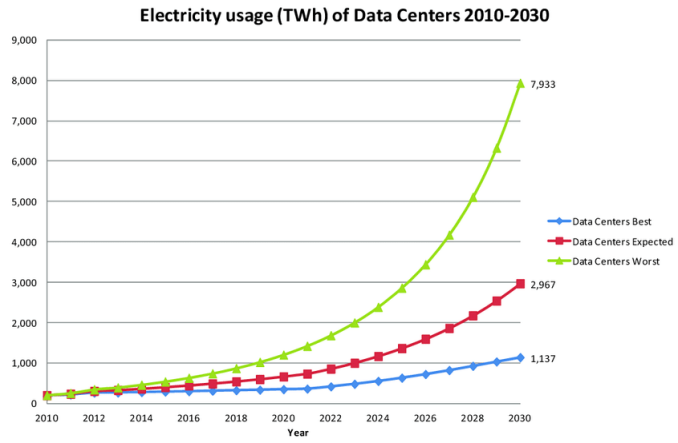
- **LSTMs (Long Short-Term Memory)**: cuentan con puertas de memoria donde deciden qué información se mantiene o se descarta
- **GRUs**: son similares a las LSTMs pero cuentan con una estructura más simple y eficiente

5. Consumo energético y sostenibilidad

El entrenamiento de las redes neuronales clásicas, sobre todo en *deep learning*, requiere una gran cantidad de recursos computacionales. Este proceso implica un consumo energético muy alto, que no solo supone el incremento de los costos operativos, sino que también contribuye significativamente a la huella de carbono de los sistemas de inteligencia artificial.

Modelos de gran escala, como los utilizados en procesamiento de lenguaje natural o visión por computadora, demandan potentes infraestructuras hardware (como GPUs o TPUs), con miles de horas de cómputo. Esta situación ha despertado una creciente preocupación en torno a la sostenibilidad del desarrollo de IA.

Figura 5. Predicción de consumo energético en los próximos años



Nota: en esta imagen podemos ver desde el consumo energético actual hasta una predicción del consumo energético estimado en los próximos años.

Para mitigar estos efectos, se han impulsado estrategias como el uso de hardware más eficiente (ASICs, FPGAs), técnicas de optimización de modelos (*pruning*, cuantización) o el uso de infraestructura energética sostenible, como centros de datos alimentados por energías renovables.

Estas limitaciones energéticas y de escalabilidad en los modelos clásicos han impulsado la búsqueda de soluciones más eficientes, entre las que destaca la computación cuántica como una alternativa prometedora para superar los retos actuales del aprendizaje automático, sobre todo, disminuyendo drásticamente el número de horas de cómputo necesarias para el entrenamiento de los modelos.

6. Computación cuántica

La computación cuántica es un campo multidisciplinario que aúna aspectos de ciencias de la computación, física y matemáticas, y utiliza principios de la mecánica cuántica para resolver problemas complejos más rápido que las computadoras clásicas. Esta capacidad resulta especialmente prometedora en campos como la biología computacional, donde tareas como la predicción de estructuras proteicas o la estimación de estabilidad frente a mutaciones

requieren una gran capacidad de cómputo que podría beneficiarse de la aceleración cuántica.

Dentro del hardware cuántico, podemos encontrar las computadoras cuánticas, las cuales son ordenadores que usan la mecánica cuántica para procesar información. Son capaces de resolver algunos tipos de problemas con más rapidez que las computadoras clásicas utilizando los principios de superposición, entrelazamiento cuántico, la interferencia cuántica y la medición. Estos ordenadores, a diferencia de los ordenadores cuánticos que funcionan con bits binarios, funcionan con qubits, los cuales pueden estar en 2 estados a la vez gracias al principio de superposición.

La superposición es un principio fundamental de las mecánicas cuánticas que les permite a los sistemas cuánticos existir en una combinación de múltiples estados simultáneamente hasta que son medidos. En la computación cuántica, eso le permite a un qubit representar tanto el estado 0 como el estado 1 o cualquier combinación lineal de estos, gracias a lo cual, el qubit puede realizar múltiples cálculos en paralelo, lo que les otorga a las computadoras cuánticas su gran potencia.

Continuando con estas propiedades cuánticas, tenemos el entrelazamiento, el cual es un fenómeno cuántico, sin equivalente clásico, en el cual los estados cuánticos de dos o más objetos se deben describir mediante un estado único que involucra a todos los objetos del sistema, aun cuando los objetos están separados espacialmente. Este fenómeno es crucial para la transmisión de información cuántica a través de protocolos como la teleportación cuántica. En el entrelazamiento perfecto, la medición de un qubit afecta instantáneamente el estado del otro.

Ahora, hablaremos sobre la medición, el proceso que ocurre al medir un qubit, donde el estado de este colapsa a uno de los valores posibles con cierta probabilidad. Este colapso establece el límite de la cantidad de información que se puede obtener de un qubit sin perturbarlo. La medición en diferentes bases permite extraer distintas informaciones del qubit, lo que es explotado en algoritmos de corrección de errores y protocolos de distribución cuántica de claves.

Para finalizar con las propiedades cuánticas que sostienen este campo, tenemos la interferencia cuántica. Esta propiedad, es el comportamiento intrínseco de un qubit, debido a la superposición, de influir en la probabilidad de que colapse de una u otra forma. Los equipos cuánticos están diseñados y creados para reducir la interferencia tanto como sea posible y asegurar los resultados más precisos. La inferencia cuántica es la base de muchos algoritmos cuánticos como el algoritmo de Grover, por ejemplo.

Una vez visto el hardware cuántico y las propiedades cuánticas que lo sustentan, vamos a ver la definición de software cuántico. Este software es el conjunto de algoritmos, lenguajes de programación, *frameworks* y herramientas diseñadas para programar, simular y ejecutar operaciones en computadoras cuánticas. Dicho software permite controlar tanto simuladores como hardware cuántico real.

6.1. Qubits y operaciones básicas

Un qubit es la unidad fundamental de información en computación cuántica. Los estados de un qubit pueden representarse mediante un vector de Hilbert:

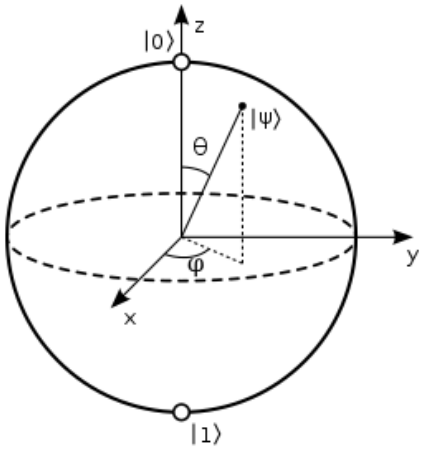
$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$$

Donde:

- $|0\rangle$ y $|1\rangle$ son los estados base
- α y ß son coeficientes complejos (con $|\alpha^2| + |\beta^2| = 1$).

A diferencia de un bit clásico que solo puede estar en un estado a la vez, un sistema cuántico con n qubits puede representar 2^n estados simultáneamente. Esto es esencial para algoritmos cuánticos como el de Grover (búsqueda) y el de Shoer (factorización) que pueden resolver problemas de manera exponencialmente más rápida que los algoritmos clásicos.

Figura 6. Estructura de un qubit



Nota: en esta imagen podemos apreciar la estructura de un qubit, la cual difiere bastante de un bit clásico.

Los qubits pueden tener estados puros y mixtos. Estos estados puros pueden representarse mediante vectores de Hilbert, pero, en cambio, los qubits mixtos son una mezcla estadística de varios estados puros, por lo que se describen utilizando una matriz de densidad que encapsula la probabilidad de encontrar el sistema en diferentes estados puros:

$$p = \sum_i p_i \, |\psi_i\rangle \, \langle\psi_i|$$

Donde p_i son probabilidades con $\sum_i p_i = 1$.

A diferencia de un estado puro (que representa máxima coherencia cuántica), un estado mixto incorpora incertidumbre clásica sobre el estado real del sistema. Estos pueden ser causados por falta de información (ignorancia clásica), interacción con el ambiente (decoherencia cuántica) o subsistemas de un estado entrelazado.

Estos estados se pueden distinguir calculando la pureza ($\mathrm{Tr}(\rho^2)$). Los estados mixtos por decoherencia son el principal obstáculo para mejorar la escalabilidad de las computadoras cuánticas, pero son fundamentales para

entender los sistemas cuánticos reales (no ideales), y dominan en entornos ruidosos, lo que los hace esenciales para técnicas como *quantum error correction* o *error mitigation*, mientras que los estados puros son ideales para algoritmos cuánticos.

Esta representación de información en estados cuánticos resulta especialmente útil en problemas biomoleculares, donde las configuraciones posibles de una proteína mutada pueden ser vastas y difíciles de explorar con enfoques clásicos. La capacidad de representar múltiples configuraciones simultáneamente podría abrir nuevas posibilidades en la predicción de propiedades bioquímicas, como la energía libre de plegamiento o ΔΔG.

6.2. Operaciones básicas: Puertas lógicas y circuitos cuánticos

Los qubits se manipulan mediante operaciones cuánticas las cuales utilizan compuertas cuánticas para transformar los qubits. Son el equivalente a las puertas lógicas en computación clásica. Estas puertas lógicas cuánticas son reversibles, a diferencia de las clásicas. Las puertas cuánticas más utilizadas operan con uno o dos qubits.

Las puertas cuánticas suelen representarse como matrices. Una puerta se puede representar como 2^k x 2^k y el número de qubits de entrada y salida han de ser iguales. El resultado de estas puertas se calcula multiplicando la matriz que representa dicha puerta por el vector que representa el estado cuántico. A continuación, veremos las puertas cuánticas más usadas:

- **Puerta de Hadamard (H):** Esta puerta opera sobre un único qubit. Crea una superposición sobre un estado base
- **Pauli-X/Y/Z(NOT cuántico):** Esta puerta opera sobre un único qubit. Gira el qubit 180º alrededor de los ejes X/Y/Z. Es el equivalente al NOT en las puertas lógicas clásicas.
- **CNOT:** Permite la correlación entre dos qubits y es fundamental para la creación de entrelazamiento

Los circuitos cuánticos se construyen mediante la aplicación secuencial de puertas cuánticas sobre un conjunto de qubits. El modelo de circuitos cuánticos es el modelo más estudiado y utilizado en la actualidad. Este modelo es el análogo al de las puertas lógicas en la computación clásica, pero aprovecha los fenómenos cuánticos de superposición y entrelazamiento para lograr mayor eficiencia computacional.

6.3. Ventajas teóricas de la computación cuántica

La computación cuántica ofrece ventajas importantes respecto a la computación clásica gracias a su capacidad de procesar información de manera más eficiente en ciertos problemas. Entre las principales ventajas se encuentran:

- **Paralelismo cuántico:** gracias al principio de superposición cuántica, un sistema de n qubits puede representar simultáneamente 2^n estados simultáneamente, lo que permite explorar una gran variedad de soluciones al mismo tiempo.
- **Aceleración exponencial:** algunos algoritmos cuánticos como el de Shor para la factorización de números primos o el de Grover para la búsqueda en bases de datos no estructurados, presentan mejoras exponenciales o cuadráticas en la velocidad de procesamiento en comparación con sus contrapartes de computación clásica.

Actualmente, se está investigando cómo aprovechar el paralelismo cuántico para acelerar la predicción de estabilidad proteica y el diseño de nuevas variantes funcionales mediante algoritmos cuánticos específicos o modelos híbridos de aprendizaje automático cuántico.

6.4. Limitaciones prácticas

A pesar de su potencial, la computación cuántica actualmente se enfrenta a desafíos tecnológicos y físicos que limitan su implementación a gran escala. Algunos de estos desafíos son:

- **Decoherencia cuántica:** la interacción de los qubits con el entorno provoca la pérdida de coherencia en los qubits, afectando la estabilidad de los cálculos cuánticos.
- **Ruido cuántico:** las imprecisiones en la manipulación de los qubits generan errores que pueden afectar a la fidelidad de los resultados.
- **Escalabilidad:** la escalabilidad de los sistemas cuánticos es una de las principales limitaciones de estos sistemas, ya que construir estos sistemas con un gran número de qubits interconectados sin aumentar los errores es un reto significativo.
- **Corrección de errores cuánticos:** la corrección de errores cuánticos es todo un reto, ya que se requieren sofisticados códigos de corrección de errores para mitigar los efectos del ruido y la decoherencia, lo que añade complejidad a los sistemas cuánticos.

A pesar de estas limitaciones que hemos comentado, los avances en tecnologías cuánticas y en algoritmos de corrección de errores están permitiendo un desarrollo continuo en la búsqueda de computadoras cuánticas más robustas y escalables.

7. Quantum Machine Learning (QML) y aplicaciones

El *machine learning* cuántico (QML) es un campo emergente que busca aplicar algoritmos cuánticos para mejorar el rendimiento de la eficiencia y la capacidad de modelos de aprendizaje automático. El QML tiene diversas aplicaciones en diferentes áreas del aprendizaje automático y la optimización. Algunas de sus aplicaciones más destacadas incluyen:

- **Clasificación cuántica:** es el uso de circuitos cuánticos parametrizados para la clasificación de datos en diferentes categorías, con potencial para reducir la complejidad de modelos clásicos.
- **Regresión cuántica:** es la aplicación de estados cuánticos y medidas probabilísticas para aproximar funciones y realizar predicciones sobre conjuntos de datos.
- **Optimización de parámetros:** es el uso de algoritmos cuánticos para mejorar procesos de optimización en modelos de *machine learning*, reduciendo así los tiempos de entrenamiento y mejorando la convergencia de estos modelos.

A medida que la tecnología cuántica avanza, se espera que QML desempeñe un papel clave en la resolución de problemas complejos en ciencia de datos, finanzas y biomedicina.

7.1. Redes neuronales cuánticas (QNNs): Concepto y teoría

Las QNNs son un tipo de modelo de *machine learning* que aprovecha los principios de la computación cuántica para realizar tareas de aprendizaje automático. Estas redes surgen de la motivación de combinar las redes neuronales con la capacidad computacional de la computación cuántica. En dichas redes se combina la capacidad de las redes neuronales tradicionales con la potencia de la computación cuántica, permitiéndoles realizar operaciones en espacios de alta dimensionalidad de manera eficiente. Este tipo de operaciones son especialmente útiles cuando se trabaja con datos complejos, como los que describen características estructurales o bioquímicas de proteínas.

Las redes neuronales clásicas tienen una gran potencia, pero a medida que los modelos se vuelven complejos, también lo hacen los problemas relacionados con el procesamiento y la optimización de espacios de alta dimensionalidad. La computación cuántica ofrece ventajas en la manipulación de altas dimensiones, lo que puede mejorar la eficiencia de las

redes neuronales en tareas complejas como clasificación de datos y optimización.

Las QNNs, gracias a los qubits, pueden representar múltiples estados a la vez, algo que las redes clásicas no pueden alcanzar. Esto permite reducir el tiempo de entrenamiento drásticamente y mejorar la eficiencia en ciertos problemas, como la búsqueda de patrones en grandes cantidades de datos. Además, el uso de operaciones cuánticas puede acelerar potencialmente el proceso de entrenamiento mediante el uso de algoritmos cuánticos, lo que hace que la optimización de redes neuronales sea más efectiva.

7.2. Diferencias clave con redes neuronales clásicas

Existen distintas diferencias fundamentales entre las RNN clásicas y las QNNs. Algunas de las diferencias más importantes son:

- **Codificación de datos cuánticos (embeddings cuánticos):**

  En las QNNs, la codificación de los datos es realizada de forma diferente respecto a las redes neuronales clásicas. Mientras que en las redes clásicas los datos se introducen en estas de forma lineal, en las QNNs se emplean tácticas de embedding cuántico, donde los datos clásicos se transforman en sus equivalentes cuánticos mediante un proceso de que permite representar la información en un espacio de alta dimensionalidad.

- **Capas parametrizadas**

  En las capas de las QNNs, en lugar de utilizar las típicas funciones de activación y capas totalmente conectadas, se emplean puertas cuánticas parametrizadas. Estas puertas cuánticas actúan como unidades de procesamiento cuántico que manipulan los estados cuánticos a través de un conjunto de parámetros que se ajustan durante el entrenamiento.

Las capas de puertas clásicas permiten que la red procese la información de una forma totalmente diferente a las redes neuronales clásicas.

- **Medición**

Un aspecto clave en las QNNs es el proceso de medición, ya que los estados cuánticos sólo se conocen cuando se mide un qubit. En las redes neuronales clásicas, los datos se procesan de forma determinística, mientras que en las QNNs es de forma probabilística, lo que puede suponer resultados diferentes en cada ejecución. Esto introduce una capa más adicional de incertidumbre en el proceso de entrenamiento y optimización.

7.3. Arquitecturas principales de QNNs

Dentro de las arquitecturas principales de las QNNs podemos encontrar algunas que combinan elementos de la inteligencia artificial clásica con operaciones cuánticas, permitiendo explorar espacios de solución de manera más eficiente en ciertos problemas. A continuación, se describen algunas de las principales arquitecturas utilizadas actualmente en el diseño de QNNs.

- **Circuitos cuánticos variacionales (VQC)**

VQC es una arquitectura que combina el procesamiento cuántico con la optimización clásica. Esta arquitectura utiliza principalmente circuitos cuánticos con puertas ajustables que se encargan de transformar los datos cuánticos de una forma eficiente. Mediante un bucle iterativo, un optimizador clásico ajusta los parámetros del circuito cuántico buscando minimizar la función de coste. Este enfoque es especialmente útil en los dispositivos cuánticos actuales (NISQ), donde las limitaciones de ruido son el principal impedimento para el uso de algoritmos cuánticos más profundos.

- **Redes neuronales cuánticas profundas (Deep QNNs)**

Las *Deep QNNs* extienden los VQC mediante la incorporación de múltiples capas cuánticas secuenciales, formadas por *embeddings*, puertas cuánticas parametrizadas, entrelazamiento y medición. Esta estructura mejora la capacidad de representación de la red, permitiendo modelar relaciones más complejas en los datos, como ocurría en el caso de las redes neuronales profundas clásicas. Sin embargo, su entrenamiento es mucho más sensible al ruido, la decoherencia y la atenuación del gradiente, lo que requiere técnicas avanzadas de optimización cuántica. A pesar de sus retos, estas redes muestran un gran potencial en aplicaciones como reconocimiento de patrones cuánticos y simulación de sistemas físicos.

8. Estado actual y desafíos de las QNNs

Como hemos visto en apartados anteriores, las QNNs son muy prometedoras dentro de la computación cuántica y el aprendizaje automático. A continuación, se presentan las implementaciones más recientes, las herramientas utilizadas en el campo y los principales obstáculos que deben superarse para su adopción a gran escala

A pesar de las limitaciones del hardware cuántico actual, diversas investigaciones han logrado avances en la aplicación de QNNs en problemas concretos. Algunos de los casos de éxito más destacados incluyen:

- **Simulación de materiales:** se han desarrollado modelos de QNNs capaces de predecir propiedades moleculares con mayor precisión que los métodos clásicos, aprovechando la naturaleza cuántica de los materiales.

- **Reconocimiento de patrones:** se han implementado QNNs para la clasificación de imágenes en espacios de alta dimensionalidad, explorando ventajas en eficiencia sobre las redes neuronales clásicas.
- **Optimización financiera:** aplicaciones de QNNs en mercados financieros han demostrado su potencial en la optimización de carteras y la detección de patrones en datos de series temporales.

8.1. Frameworks y herramientas prácticas

El desarrollo de QNNs requiere el uso de herramientas especializadas que permitan la simulación y ejecución en hardware cuántico real. Actualmente, existen diversos *frameworks* que facilitan la implementación de modelos cuánticos:

- **Qiskit (IBM Quantum):** biblioteca de código abierto para la creación y simulación de circuitos cuánticos, con soporte para algoritmos de machine learning cuántico.
- **PennyLane (Xanadu):** framework que permite la integración de QNNs con librerías de deep learning clásicas como PyTorch y TensorFlow, facilitando el desarrollo de modelos híbridos.
- **TensorFlow Quantum (Google):** extensión de TensorFlow para el entrenamiento de modelos cuánticos dentro de entornos de machine learning clásicos.
- **Cirq (Google):** herramienta enfocada en la creación de circuitos cuánticos optimizados para hardware específico, utilizada en aplicaciones experimentales.

8.2. Desafíos técnicos

A pesar de los avances recientes, las QNNs enfrentan desafíos que limitan su implementación en problemas reales:

- **Hardware NISQ:** Las computadoras cuánticas actuales operan en un régimen de "escala intermedia ruidosa" (NISQ), lo que implica errores significativos y un número limitado de qubits disponibles.
- **Falta de datasets cuánticos:** La mayoría de los conjuntos de datos actuales están diseñados para modelos clásicos, lo que dificulta la generación de datos específicamente optimizados para el aprendizaje cuántico.
- **Entrenamiento estable:** La variabilidad en la medición cuántica y la atenuación del gradiente dificultan la convergencia de los modelos, requiriendo técnicas de optimización especializadas.

8.3. Futuras direcciones

Para que las QNNs alcancen su máximo potencial, es necesario abordar diversos frentes de investigación y desarrollo:

- **QNNs tolerantes a errores:** Se está explorando la posibilidad de diseñar modelos robustos a la decoherencia y al ruido inherente en los dispositivos cuánticos.
- **Integración con IA clásica:** La combinación de redes neuronales cuánticas con técnicas tradicionales de *deep learning* podría mejorar su aplicabilidad en problemas reales.
- **Aplicaciones industriales:** A medida que el hardware cuántico mejora, se espera que las QNNs se apliquen en áreas como la biomedicina, la logística y la inteligencia artificial avanzada.

9. Biotecnología. Inteligencia artificial y computación cuántica en este campo.

Una vez vistos los principales conceptos sobre IA y computación cuántica, veremos como estas tecnologías han sido capaces de influenciar enormemente el ámbito de la biotecnología. Pero, antes, veremos por encima qué es la biotecnología y como está relacionada con el problema que vamos a abordar en este TFM

9.1. Biotecnología

La biotecnología es un campo multidisciplinar que incorpora principios de la biología, química, ingeniería e informática. La biotecnología se centra en desarrollar diversos productos, procesos y soluciones que aporten a fines médicos, industriales o medioambientales. Este campo se compone diferentes áreas como la ingeniería genética, la biología molecular, la bioinformática y la biología sintética.

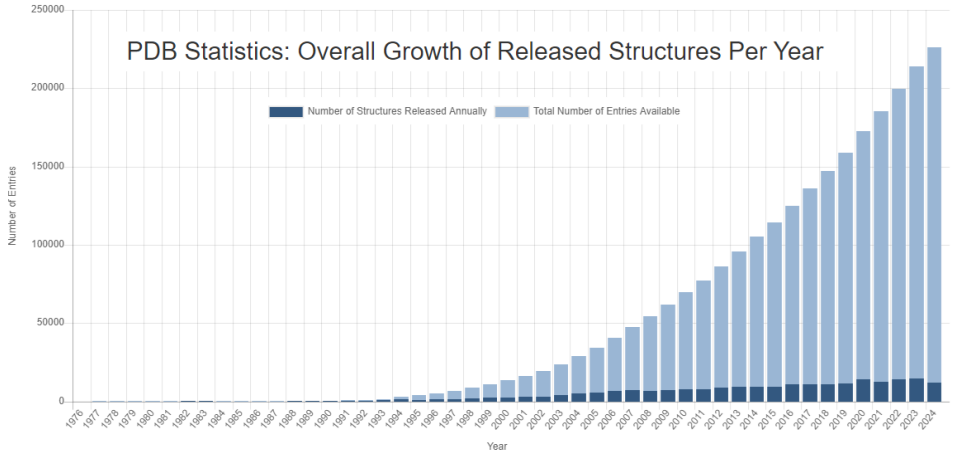
9.2. Inteligencia artificial en la biotecnología

Como se viene comentando a lo largo de todo este apartado, la inteligencia artificial ha supuesto una gran revolución para este campo, permitiendo procesar grandes cantidades de datos y permitiendo crear modelos a partir de estos con los cuales es posible resolver problemas que requerirían años, sino décadas de investigación para acercarse a una aproximación de dicha solución. Entre sus aplicaciones más destacadas se encuentra la predicción de estructuras de proteínas, como ha demostrado AlphaFold, que ha transformado la biología estructural en apenas unos años. También se emplea en la detección de mutaciones funcionales y en la predicción de cómo afectan a la estabilidad o función de una proteína, lo cual resulta esencial en investigación biomédica y diseño de fármacos.

Además, la IA facilita la identificación de secuencias genómicas relevantes, el análisis de redes metabólicas y el diseño de nuevas moléculas o enzimas, abriendo la puerta a la ingeniería de proteínas y a terapias más personalizadas. Estas capacidades han hecho que procesos antes experimentales y muy costosos puedan resolverse ahora en tiempos mucho más cortos y con menor inversión de recursos.

Uno de los ejemplos más conocidos de este avance es AlphaFold, desarrollado por DeepMind, capaz de predecir con gran precisión la forma tridimensional de las proteínas. Su impacto ha sido tan grande que el número de estructuras conocidas ha crecido exponencialmente en los últimos años, acelerando enormemente la investigación en este ámbito (*Figura 6*).

Figura 7. Número de estructuras de proteínas descubiertas en los últimos años



Nota: en esta imagen podemos ver el avance en el descubrimiento de estructuras de proteínas y como este se ha visto increíblemente afectado por el desarrollo de AlphaFold, sobre todo en los últimos años. (AlphaFold, 2025)

9.3. Computación cuántica en la biotecnología

La computación cuántica supone un avance disruptivo con un gran potencial para ayudar a superar las limitaciones actuales de la bioinformática. Algunas de estas limitaciones son la escalabilidad cuando se deben simular sistemas biológicos de gran complejidad, como proteínas de gran tamaño, interacciones entre proteínas o interacciones moleculares. La gran cantidad de posibles configuraciones disponibles hace que estos problemas requieran una grandísima capacidad computacional.

La respuesta a esto puede encontrarse en los principios de superposición y entrelazamiento, los cuales pueden permitir explorar diversos estados moleculares de forma simultánea. Esto ha supuesto que se abra la posibilidad de realizar simulaciones mucho más precisas de sistemas biológicos complejos, reduciendo el tiempo necesario de simulación drásticamente.

Entre las aplicaciones más prometedoras de la computación cuántica en biotecnología destacan:

- **Predicción energética de mutaciones en proteínas**, permitiendo entender mejor cómo cambios puntuales en la secuencia afectan a la estabilidad y función de la proteína.
- **Diseño de nuevos fármacos y optimización de interacciones proteína-ligando**, mediante simulaciones cuánticas capaces de modelar con mayor fidelidad la dinámica molecular.
- **Modelado de rutas metabólicas y redes biológicas complejas**, donde la computación cuántica podría explorar configuraciones que resultan intratables para algoritmos clásicos.
- **Simulación de materiales y biomoléculas**, lo que abre nuevas posibilidades en biología sintética e ingeniería de proteínas.

Aunque, como hemos visto anteriormente, la computación cuántica todavía se encuentra en un estado de desarrollo muy temprano, ya existen estudios y prototipos que muestran como este campo puede ayudar enormemente a encontrar soluciones a este tipo de problemas biológicos, y más teniendo en cuenta la combinación de la computación cuántico junto con la inteligencia artificial.

9.4. Avances recientes, estado actual y futuro

La IA ha supuesto el impacto más notorio en este campo gracias a los modelos de AlphaFold y RoseTTAFold, que han permitido millones de estructuras de proteínas en tiempo récord, algo impensable hasta hace poco. Estos avances han acelerado investigaciones en biología estructural, diseño de fármacos y biología sintética, facilitando el acceso a información fundamental para comprender procesos moleculares.

Por su parte, la computación cuántica, aunque aún en una fase inicial, ya ha mostrado resultados alentadores en problemas biomoleculares concretos. Algunos prototipos han demostrado capacidad para simular interacciones proteína-ligando, estimar energías de enlace en pequeñas moléculas y explorar configuraciones moleculares en espacios de alta dimensionalidad que resultan intratables para los métodos clásicos. Si bien estas aplicaciones se limitan hoy en día a sistemas de pequeña escala y a entornos de simulación,

representan un primer paso hacia la validación práctica del potencial cuántico en biotecnología.

El estado actual puede definirse como un escenario híbrido: la IA es ya una herramienta consolidada en biología computacional, mientras que la computación cuántica está en fase exploratoria, con experimentos prometedores, pero todavía condicionados por el ruido, la decoherencia y la disponibilidad limitada de hardware cuántico fiable.

Mirando hacia el futuro, se espera que la sinergia entre ambas tecnologías dé lugar a un cambio de paradigma. Los modelos híbridos, que combinan algoritmos clásicos con módulos cuánticos especializados, se perfilan como la vía más realista en el corto plazo, especialmente en campos como la predicción de mutaciones funcionales, la optimización de redes metabólicas y la simulación de sistemas biomoleculares complejos.

Capítulo 4: Marco Metodológico

En este capítulo entraremos en detalle en el conjunto de datos seleccionado y sus principales características, la metodología que se ha seguido para abordar el problema de investigación y las herramientas utilizadas durante el proceso de desarrollo del proyecto. En este apartado se ha expuesto el enfoque metodológico adoptado para implementar y comparar redes neuronales clásicas y cuánticas en un contexto real de predicción de estabilidad de mutaciones proteicas.

Además, se describe el flujo de trabajo que se ha seguido, desde el planeamiento del problema y detección de posibles limitaciones y retos técnicos hasta la selección de *datasets* relevantes, el diseño experimental y la validación de los resultados obtenidos.

Este capítulo, también presenta un registro y justificación del tipo de investigación escogida, así como las decisiones técnicas y metodológicas tomadas durante el proyecto. Con este apartado se busca proporcionar una visión clara, rigurosa y reproducible del proceso seguido en el desarrollo de este TFM.

1. Población y muestra

En este proyecto se ha trabajado con *dataset* s2648, uno de los conjuntos de datos más populares en lo que refiere a estabilidad de proteínas ante mutaciones puntuales. El conjunto de datos s2648 es uno de los *benchmarks* más utilizados en el ámbito de la bioinformática estructural centrada en el estudio de la estabilidad de proteínas ante mutaciones puntuales. Fue desarrollado con el objetivo de proporcionar una base de datos experimentalmente validada y curada que permita evaluar la capacidad predictiva de distintos modelos.

Este *dataset* se ha utilizado en numerosos estudios científicos como referencia para comparar modelos de predicción de estabilidad y se ha convertido en un

recurso estándar en investigaciones relacionadas con el diseño racional de proteínas, modelado energético y evaluación de mutaciones.

Este conjunto de datos contiene un total de 2.648 registros, cada uno correspondiente a una mutación puntual en una proteína conocida. La propiedad que utilizaremos como variable objetivo en este conjunto de datos es ΔΔG, el cambio en la energía libre de Gibbs, una métrica para predecir cómo una mutación puntual afectará a la estabilidad de una proteína. Este valor cuantifica el efecto de la mutación sobre la estabilidad de la proteína: si $\Delta\Delta G < 0$, la mutación es estabilizante; si $\Delta\Delta G > 0$, es desestabilizante.

El conjunto de datos está estructurado en formato CSV y contiene las siguientes columnas:

- **CLID**: Identificador de cadena (Proteína + Chain ID).
- **PDB**: Código de estructura cristalográfica.
- **CHAIN**: Cadena de la proteína.
- **MUT**: Notación de la mutación (por ejemplo, H32Y).
- **DDG**: Valor experimental de ΔΔG (variable objetivo para la regresión).
- **pH**: Condición de pH durante el experimento.
- **T**: Temperatura experimental en grados Celsius.

Cada fila del *dataset* representa un experimento en el cual se ha evaluado el efecto de una mutación puntual sobre la estabilidad de una proteína. Las columnas pH y T permiten modelar el contexto experimental en el que se llevó a cabo la medición de ΔΔG, lo cual es esencial para desarrollar modelos más realistas y generalizables.

La elección de este *dataset* se basa principalmente, como ya se ha comentado, en la popularidad de este como *benchmark* en diversos experimentos para la validación de nuevas metodologías. El hecho de que los datos que contiene se encuentran curados y listos para trabajar con ellos, facilita bastante el trabajo y ha inclinado bastante la balanza en su favor a la hora de elegir *dataset*.

También influye el que posee una estructura clara e incluye información experimental como el pH y la temperatura (T) y tiene un tamaño de datos adecuado para entrenar redes neuronales y ejecutar simulaciones de redes neuronales cuánticas sin requerimientos computacionales excesivos.

2. Software utilizado

Para el desarrollo de este proyecto se han empleado diversas herramientas de software, *frameworks* y entornos de ejecución que permiten la implementación, entrenamiento y evaluación de modelos tanto clásicos como cuánticos. En los siguientes subapartados, se detallan las principales tecnologías utilizadas.

2.1. Google Colab

Google Colab es una plataforma que permite y facilita la ejecución y el desarrollo de código en la nube. Colab ofrece un entorno de ejecución basado en Jupyter Notebook, lo que facilita la organización del trabajo en celdas, permite la integración de visualizaciones, y mejora la trazabilidad del proceso experimental. Además, esta plataforma permite el acceso a recursos hardware como CPUs, GPUs y TPUs y tiene una alta compatibilidad con bibliotecas como TensorFlow, Scikit-learn y Qiskit.

Todas estas características anteriores hacen a Google Colab una de las mejores herramientas actuales para el desarrollo de modelos de aprendizaje automático.

Ha sido el entorno más utilizado a lo largo de este proyecto y donde se han realizado la ejecución de la mayoría de los modelos.

2.2. Visual Studio Code

Visual Studio Code es un editor de código desarrollado por Microsoft para Windows, Linux, macOS y Web. Incluye soporte para la depuración, resaltado de sintaxis, finalización inteligente de código y refactorización de código. Es software gratuito y de código abierto, aunque la descarga oficial está bajo software privativo e incluye características personalizadas por Microsoft.

En este proyecto se ha utilizado VS Code exclusivamente para algunas de las ejecuciones más largas de las QNNs, ya que, en mi caso, la CPU personal era más potente que la de Google Colab y esto aceleró el entrenamiento de las QNNs, las cuales llegaron a tener ejecuciones que tardaron días.

2.3. TensorFlow

TensorFlow es una librería de código abierto desarrollada por Google para la creación y entrenamiento de modelos de aprendizaje automático, con especial énfasis en redes neuronales. Su arquitectura flexible permite desde modelos sencillos hasta estructuras más complejas, lo que la hace útil tanto para principiantes como para investigadores. Además, ofrece soporte para aceleradores de hardware (GPUs y TPUs) sin necesidad de modificar el código, y es compatible con múltiples plataformas, lo que facilita la ejecución en entornos locales o en servidores.

En este proyecto se ha utilizado esta librería para el desarrollo de la Fully Connected Neural Network (FCNN) creada desde cero, permitiendo ajustar su arquitectura, funciones de pérdida y algoritmos de optimización.

2.4. Scikit-learn

Scikit-learn es una biblioteca de Python de código abierto ampliamente utilizada para el aprendizaje automático. Es un marco integral que proporciona una amplia gama de herramientas y algoritmos para realizar diversas tareas de ciencia de datos, incluyendo preprocesamiento de datos, clasificación, regresión, agrupación y reducción de dimensionalidad.

En este proyecto scikit-learn se ha empleado principalmente como herramienta de apoyo para el preprocesamiento de los datos, la división en conjuntos de entrenamiento y pruebas y la evaluación de los modelos. Su integración fluida con otras librerías científicas y su enfoque modular han hecho de Scikit-learn una herramienta esencial en la fase de análisis de datos inicial y en la fase de validación del modelo.

2.5. Qiskit

Qiskit es un *framework* de código abierto desarrollado por IBM que permite diseñar y ejecutar algoritmos de computación cuántica. Está orientado tanto a la investigación como a la aplicación práctica, ya que ofrece herramientas para construir circuitos cuánticos, parametrizarlos y simularlos en entornos controlados o ejecutarlos en hardware cuántico real. Su flexibilidad lo convierte en uno de los entornos más utilizados actualmente en el campo de la computación cuántica.

Una de las principales ventajas de Qiskit respecto a otros *frameworks* es su madurez y comunidad activa, lo que garantiza documentación extensa, soporte y trabajo y actualizaciones constantes en el *framework*. Un ejemplo de esto es que, durante el desarrollo del proyecto, Qiskit ha pasado de la versión 1,4.1 a la 1.4.4. Además, ofrece gran interoperabilidad con librerías clásicas de *machine learning* en Python, lo que facilita la construcción de modelos híbridos (clásico-cuánticos).

Otra cualidad importante es que permite trabajar con distintos niveles de abstracción: desde la definición detallada de circuitos cuánticos hasta el uso de módulos de alto nivel para aplicaciones en optimización, química cuántica o aprendizaje automático. En comparación con alternativas como Cirq o PennyLane, Qiskit destaca por su enfoque integral y por disponer de un simulador cuántico estable y eficiente (*StatevectorSimulator*), que permite ejecutar experimentos de manera exacta sin los problemas de ruido del hardware actual.

En este proyecto se ha utilizado Qiskit para la implementación de la red neuronal cuántica, utilizando **StatevectorEstimator,** que posibilita un entrenamiento exacto sin depender del hardware cuántico físico, evitando así los problemas de ruido y SparsePauliOp para definir observables, los cuales nos permiten calcular valores esperados como salida del modelo.

2.6. Qiskit Machine learning

Qiskit Machine Learning es un módulo especializado dentro del ecosistema de Qiskit que proporciona herramientas para integrar modelos de *machine learning* con algoritmos cuánticos. Su propósito es facilitar la construcción de redes neuronales cuánticas (QNNs), modelos variacionales y sistemas híbridos cuántico-clásicos, permitiendo a los investigadores y desarrolladores explorar aplicaciones del aprendizaje automático en un contexto cuántico.

Este módulo destaca porque ofrece una interfaz directa entre los circuitos cuánticos construidos con Qiskit y entornos de aprendizaje automático clásicos como PyTorch y TensorFlow. Gracias a conectores como TorchConnector, es posible entrenar modelos cuánticos utilizando los mismos optimizadores, funciones de pérdida y rutinas de *back propagation* que se usan en modelos clásicos, lo que facilita su adopción y experimentación.

En comparación con otras bibliotecas externas (por ejemplo, PennyLane), Qiskit Machine Learning tiene la ventaja de estar completamente integrado dentro de Qiskit, lo que asegura compatibilidad nativa con los simuladores y hardware cuántico de IBM. Además, su diseño modular permite trabajar tanto con circuitos sencillos como con arquitecturas más complejas, ofreciendo flexibilidad sin perder eficiencia.

En este trabajo, Qiskit Machine Learning ha sido utilizado para implementar y entrenar redes neuronales cuánticas orientadas a la predicción de la estabilidad de proteínas tras mutaciones. Concretamente, se ha empleado la clase EstimatorQNN para definir modelos variacionales a partir de circuitos cuánticos personalizados, junto con observables específicos como combinaciones de operadores Pauli. Posteriormente, mediante

TorchConnector, estos modelos se integraron dentro de PyTorch, lo que permitió combinarlos con capas clásicas y aplicar técnicas habituales de entrenamiento (optimización con AdamW, *early stopping*, regularización, etc.).

Gracias a estas herramientas, fue posible construir un sistema híbrido en el que las QNNs procesaban representaciones reducidas de las características de las mutaciones y, junto con una rama clásica, generaban predicciones precisas. En resumen, Qiskit Machine Learning ha sido esencial para llevar las ideas de la computación cuántica al terreno práctico, integrándolas con los métodos de *deep learning* ya consolidados.

2.7. Otras herramientas

Además de los entornos y librerías principales de lo que se ha hablado anteriormente, este proyecto se ha apoyado en una serie de bibliotecas científicas y de *machine learning* ampliamente utilizadas en la comunidad investigadora.

Entre ellas, podemos destacar algunas como NumPy, la cual ha sido esencial para el manejo de estructuras de datos numéricas, proporcionando operaciones matriciales y funciones matemáticas optimizadas que permiten trabajar de manera eficiente con grandes volúmenes de información.

Por su parte, pandas se utilizó para la carga, limpieza y manipulación de los datos de mutaciones proteicas. Su capacidad para gestionar *dataframes* de manera flexible y con una sintaxis clara la convierte en una herramienta clave en la fase de preprocesamiento y análisis exploratorio de los datos.

El núcleo del entrenamiento clásico se apoyó en PyTorch, una de las bibliotecas más utilizadas para el desarrollo de redes neuronales. En este proyecto, PyTorch no solo facilitó la implementación de la red neuronal totalmente conectada (FCNN), sino también la integración de modelos híbridos gracias a su compatibilidad con el conector TorchConnector de Qiskit

Machine Learning, que permitió entrenar las QNNs dentro del mismo marco de trabajo.

Otras bibliotecas de gran utilidad fueron SciPy, empleada para el cálculo de métricas estadísticas como coeficientes de correlación, y scikit-learn, utilizada principalmente en el preprocesamiento de datos (escalado, PCA) y en la evaluación de modelos clásicos de referencia.

Finalmente, aunque en menor medida, Matplotlib resultó útil para la visualización de resultados y métricas, permitiendo analizar de manera gráfica la evolución del entrenamiento y las predicciones de los modelos.

En conjunto, todas estas herramientas han constituido la base técnica sobre la cual se ha construido el proyecto, garantizando tanto la eficiencia en el procesamiento de datos como la flexibilidad para experimentar con diferentes arquitecturas y enfoques.

3. Hardware utilizado

Habiendo comentado ya la parte de software utilizado, en este apartado nos centraremos en el hardware que se ha empleado en este proyecto y hablaremos sobre como podría afectar el uso de un hardware cuántico en la eficiencia y resultados de la red cuántica desarrollada. A continuación, entraremos en detalle en dicho hardware.

3.1. Hardware disponible en Google Colab

En su configuración por defecto, Colab proporciona acceso a máquinas virtuales con procesadores Intel Xeon de 2 vCPU y 13 GB de memoria RAM, aunque en la modalidad de pago (Colab Pro o Pro+) es posible obtener configuraciones más avanzadas, llegando a disponer de hasta 96 vCPU y 624 GB de RAM.

Una de las mejores opciones que facilita Colab es el acceso a aceleradores de hardware. De manera estándar, Colab asigna GPUs NVIDIA Tesla K80 con 12 GB de VRAM, aunque también pueden habilitarse Tesla T4 (16 GB de VRAM), P100 (16 GB) o incluso A100 (40 GB), aunque estas opciones son de pago.

El acceso a estos recursos, sin embargo, está condicionado por limitaciones de uso (tiempo máximo de sesión, recursos compartidos y prioridad según el plan del usuario). Aun así, para fines de prototipado, entrenamiento de modelos de tamaño medio y pruebas experimentales —como las realizadas en este proyecto—, Colab resulta una herramienta altamente eficaz y flexible.

3.2. Hardware Personal

Como hardware personal, el equipo utilizado fue un Lenovo Legion Y530-15ICH-1060, un portátil que cuenta con especificaciones competitivas para tareas de cómputo intensivo. El dispositivo dispone de un procesador Intel Core i7-8750H de 6 núcleos, acompañado de 16 GB de memoria RAM DDR4, y una GPU dedicada NVIDIA GeForce GTX 1060 con 6 GB de VRAM. Estas características resultaron particularmente útiles para entrenamientos de mayor duración, donde la GPU permitió reducir significativamente los tiempos de cómputo frente al uso exclusivo de CPU.

En el contexto de este proyecto, el uso de este hardware se destinó principalmente a ejecuciones largas de modelos híbridos clásicos-cuánticos, las cuales en Colab podían exceder las restricciones de uso. La disponibilidad de unas CPU y GPU dedicada permitió entrenar modelos que requerían horas o incluso días, asegurando estabilidad y continuidad en los procesos de entrenamiento.

3.3. Hardware cuántico

Aunque los experimentos principales se han ejecutado en simuladores de Qiskit, resulta relevante comentar el papel que tendría la ejecución en un hardware cuántico real.

En la actualidad, plataformas como IBM Quantum Experience ponen a disposición procesadores cuánticos accesibles a través de la nube. No obstante, su uso presenta limitaciones derivadas de la propia tecnología actual, como la decoherencia cuántica, el ruido en las compuertas y la escasa cantidad de qubits útiles disponibles. A esto se suma el coste asociado: por ejemplo, en IBM Cloud el precio de ejecución se sitúa en torno a 100 $/minuto, con una prueba gratuita de apenas 10 minutos al mes, mientras que en el plan gratuito las colas de espera pueden alcanzar hasta 34 días. Estas limitaciones repercuten directamente en la precisión y estabilidad de los modelos de Quantum Machine Learning, además de dificultar su utilización práctica en proyectos de investigación aplicada.

Si se hubiera contado con un hardware cuántico de última generación, la ejecución de las QNNs desarrolladas podría haberse beneficiado de una mayor fidelidad en la representación de los estados cuánticos y, potencialmente, de mejoras en la eficiencia de ciertos cálculos frente a los simuladores clásicos. No obstante, dadas las limitaciones tecnológicas actuales, el uso de simuladores ha sido la opción más viable para este trabajo, permitiendo explorar arquitecturas híbridas sin las restricciones de ruido cuántico presentes en hardware real.

4. Redes implementadas

En este punto hablaremos sobre el planteamiento del problema, el proceso y la metodología de desarrollo de los distintos modelos, así como del preprocesamiento llevado a cabo, las métricas que se buscan optimizar y la arquitectura en detalle de cada uno de estos. El código del proyecto está disponible en el siguiente enlace:

https://github.com/marrocrod/TFM_UAX

4.1. Planteamiento, proceso y metodología de desarrollo del proyecto

El planteamiento del problema comenzó con una fase de investigación en el área de la biotecnología, recurriendo tanto a literatura especializada como a profesionales del sector para adquirir el conocimiento mínimo necesario para trabajar en este proyecto. Fue necesario aprender desde cero a trabajar con proteínas: qué son, cómo se componen, qué papel juegan los aminoácidos y cuáles son sus tipos, así como las diferentes estructuras proteicas. Esta etapa inicial representó una fuerte inversión de tiempo debido a la gran curva de aprendizaje, pero resultó imprescindible, ya que cuanto mayor es la comprensión del dominio biológico, más adecuado puede ser el diseño de modelos de *machine learning* aplicados al problema, especialmente la etapa de preprocesamiento.

Una vez adquirida esta base, el siguiente paso fue la búsqueda de un conjunto de datos adecuado para comparar modelos clásicos y cuánticos. Tras explorar alternativas como ProTherm, SKEMPIv2 o conjuntos derivados de FireProtDB, se seleccionó finalmente el *dataset* s2648 por su popularidad y facilidad de uso. Posteriormente se revisaron implementaciones previas sobre este conjunto, donde la mayoría de los trabajos publicados empleaban enfoques estadísticos más que modelos de *machine learning*. Entre ellos destacó el método SAAFEC-SEQ, citado en varios artículos como uno de los mejores enfoques para este problema. Tras comparar con otras aproximaciones, se decidió implementar un primer modelo inspirado en este enfoque como *baseline* y poder comparar los resultados de los demás modelos. Su desarrollo fue relativamente sencillo, siendo el preprocesamiento y la optimización de parámetros los aspectos más trabajosos. Aun así, se logró reproducir resultados similares a los encontrados en los diversos artículos científicos en un tiempo razonable.

Con esta línea base establecida, el siguiente paso fue implementar una red neuronal totalmente conectada (FCNN) que reutilizara el mismo preprocesamiento de datos, con el objetivo de compararla posteriormente con un modelo cuántico. Para ello se utilizó scikit-learn en la preparación de los

datos (división en entrenamiento, validación y prueba, estandarización y generación de características auxiliares), y TensorFlow/Keras para la construcción y entrenamiento de la red. La fase de desarrollo de la FCNN siguió un proceso iterativo donde inicialmente se realizaron pruebas simples con validación interna y después introducir estrategias más robustas como la validación cruzada k-fold combinada complementando esta con un conjunto de prueba fijo. Una vez consolidada esta estructura de entrenamiento, se procedió a la optimización de los hiperparámetros clave de la red, lo que mejoró tanto la estabilidad como el rendimiento del modelo, hasta llegar a un punto aceptable.

El paso siguiente supuso adentrarse en otro campo completamente nuevo, la computación cuántica. Tras una primera toma de contacto con este ámbito, fue necesario elegir entre distintos frameworks (Pennylane, Cirq, Ocean SDK, entre otros), pero, finalmente se optó por Qiskit. Esto fue principalmente por su extensa documentación y comunidad activa. Esta decisión implicó una nueva curva de aprendizaje, agravada por la rápida evolución de la librería debido a lo cual muchos tutoriales y artículos, e incluso algunas herramientas de IA, se encontraban desactualizados y empleaban versiones obsoletas.

La implementación de la QNN requirió rediseñar el preprocesamiento de datos para adaptarlo al paradigma cuántico. A partir de ahí, se exploraron las distintas posibilidades de Qiskit como *backends*, *simuladores*, tipos de *ansatz*, observadores, número de qubits y dimensionalidad de entrada. Esta fase fue, sin duda, la más larga y compleja del proyecto, ya que cambios aparentemente pequeños —como variar el número de *shots*, la profundidad del circuito o el método de cálculo del gradiente— podían disparar el tiempo de entrenamiento desde minutos hasta varios días (en un caso extremo, una ejecución llegó a prolongarse durante cinco días al incrementar la profundidad de la red). Estos contratiempos, aunque inicialmente supusieron un obstáculo, formaron una parte esencial del proceso de aprendizaje, algo que facilitó el desarrollo de este modelo posteriormente.

Finalmente, tras experimentar con diferentes configuraciones y seleccionar las características de Qiskit más adecuadas para el problema, se implementó una versión inicial de la QNN que fue refinándose a lo largo de múltiples

iteraciones. Mediante este proceso de ajuste y optimización progresiva se logró un modelo híbrido cuántico-clásico con resultados satisfactorios y tiempos de entrenamiento razonables.

4.2. Métricas de evaluación

Para evaluar el rendimiento de los modelos en la predicción de los valores de ΔΔG, se han considerado varias métricas complementarias. Cada una de ellas aporta una perspectiva distinta sobre la calidad de las predicciones y resulta relevante en el contexto biológico de este proyecto:

- **Error cuadrático medio (MSE):** mide el promedio de los errores al cuadrado entre los valores experimentales y las predicciones. Penaliza con mayor peso los errores grandes, lo cual es importante dado que en mutaciones de proteínas incluso una predicción muy desviada puede cambiar la interpretación del efecto sobre la estabilidad.
- **Error absoluto medio (MAE):** cuantifica el error promedio en las mismas unidades de ΔΔG (kcal/mol). A diferencia del MSE, todos los errores contribuyen de forma lineal, lo que facilita una interpretación más directa sobre cuánto se desvía, en promedio, la predicción de la medida experimental.
- **Coeficiente de correlación de Pearson (PCC)**: evalúa la correlación lineal entre los valores predichos y los experimentales. En el contexto de predicción de estabilidad proteica, no solo importa el error absoluto, sino también la capacidad del modelo de capturar tendencias globales (ej. identificar correctamente qué mutaciones estabilizan o desestabilizan). Un PCC alto indica que el modelo refleja de forma consistente las variaciones relativas en ΔΔG.

En conjunto, estas tres métricas permiten una evaluación más robusta de los resultados ya que mientras que MSE y MAE cuantifican la magnitud de los errores, el PCC informa sobre la capacidad del modelo de capturar la relación entre los valores experimentales y las predicciones. En estos modelos se ha buscado optimizar de manera simultánea estas métricas para obtener unos

buenos resultados tanto en términos numéricos como interpretativos dentro del estudio de mutaciones en proteínas.

4.3. Preprocesamiento de datos

Todos los modelos desarrollados parten del mismo conjunto de datos, el S2648, que recoge mutaciones puntuales con sus correspondientes valores experimentales de variación de energía libre de plegamiento (ΔΔG). A partir de este *dataset* se extrajeron de forma sistemática la posición de la mutación, el aminoácido original (wild type) y el aminoácido mutado, lo que constituyó la base para la construcción de los vectores de características.

El flujo de preprocesamiento difirió según el modelo implementado:

- **Modelos clásicos (XGBoost y FCNN):**

  Además de las propiedades fisicoquímicas de los aminoácidos (volumen, hidrofobicidad, flexibilidad y tipo de mutación), se emplearon representaciones de contexto obtenidas a partir de la secuencia de la proteína. En particular, se utilizó una ventana de ±7 residuos vecinos alrededor de la posición mutada, así como los *embeddings* generados con ProtT5, un modelo de *transformer* entrenado específicamente para secuencias proteicas. Esto permitió disponer de una representación contextualizada de la proteína a nivel de secuencia primaria y estructura local.

- **Modelo cuántico (QNN):**

  El pipeline fue más ligero en comparación con los modelos clásicos. No se incluyeron embeddings ProtT5, ya que el objetivo era mantener una entrada de dimensionalidad controlada y compatible con el número de qubits disponible. En su lugar, se construyó un vector con propiedades básicas de la mutación, codificaciones one-hot de los residuos implicados y un conjunto de características derivadas de los vecinos en

la secuencia, incluyendo el cómputo de frecuencias de aminoácidos en la ventana local. Estos vectores se sometieron posteriormente a escalado y reducción de dimensionalidad mediante PCA, antes de ser integrados en el circuito cuántico.

4.4. Modelo XGBoost (SAAFEC-CEQ)

El primer modelo desarrollado en este proyecto ha sido XGBoost, un regresor basado en *boosting* de árboles de decisión, aplicando el enfoque SAAFEC-SEQ, el cual es ampliamente mencionado en diversos artículos como una de las aproximaciones más sólidas en el campo del *machine learning* para el análisis de mutaciones en proteínas en el *dataset* s2648.

Aunque la construcción del vector de entrada se ha descrito previamente, conviene destacar que en este modelo se emplea su versión más completa, enriquecida con información fisicoquímica, contexto secuencial y representaciones derivadas de ProtT5. De esta manera, el aprendizaje del modelo se apoya simultáneamente en propiedades a nivel de residuo, en el entorno local y en *embeddings* de lenguaje de proteínas de alta dimensionalidad.

Este modelo se construyó a partir de un vector de entrada enriquecido que combina tres tipos de información:

- **Propiedades fisicoquimicas** (Δvolumen, Δhidrofobicidad, flexibilidad y tipo de mutación)**:** diferencias en volumen, hidrofobicidad y flexibilidad entre el aminoácido original y el mutado, junto con un identificador del tipo de mutación.
- **Contexto secuencial:** representación explícita de los vecinos a ±7 posiciones y conteo de aminoácidos en la vecindad.
- **Embeddings ProtT5:** representaciones de alta dimensionalidad del residuo mutado extraídas de un modelo de lenguaje de proteínas.

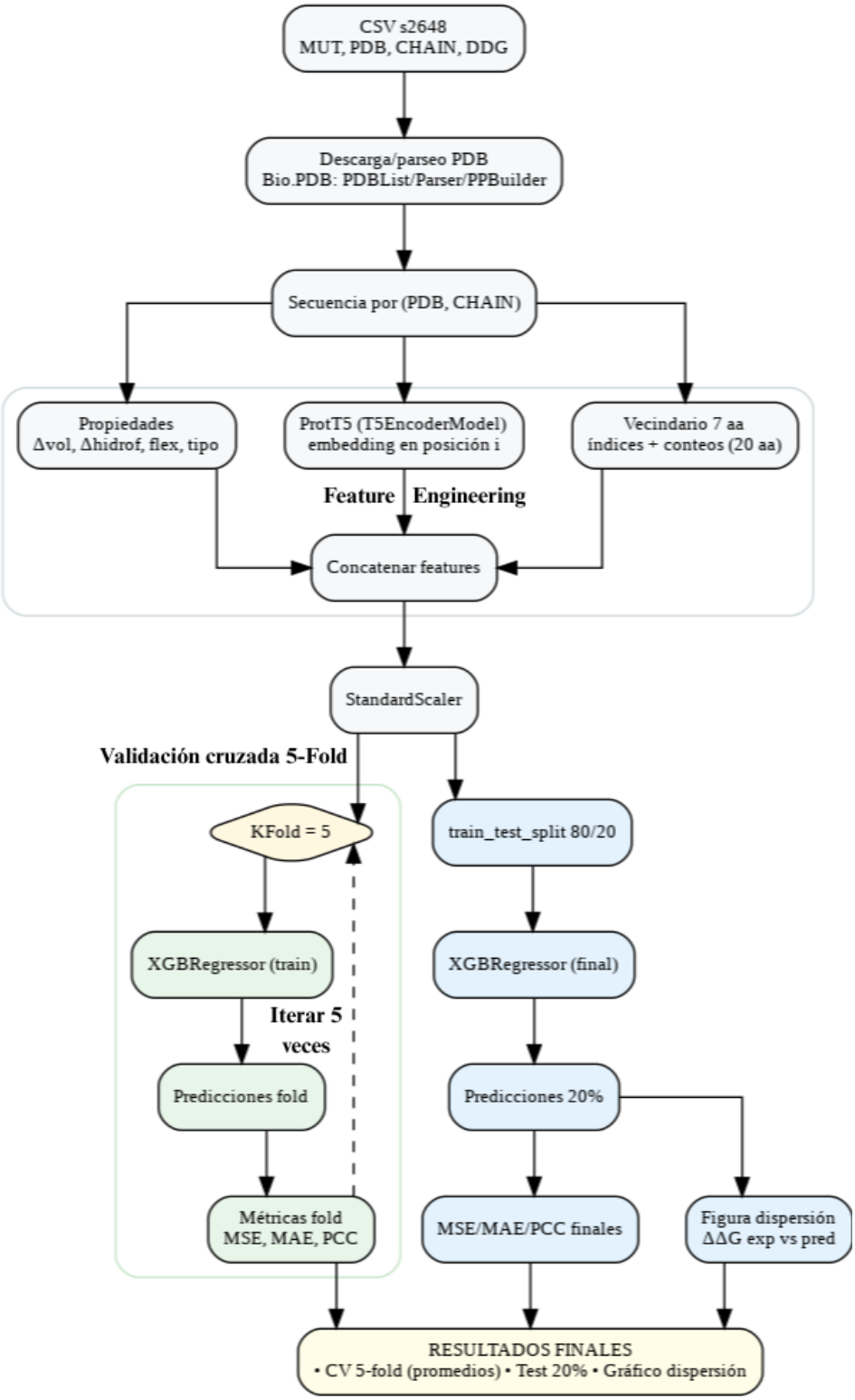
Respecto a la metodología de validación, se aplicaron 2 estrategias complementarias:

- **Validación cruzada (KFold=5)**: el conjunto de datos completo se dividió en cinco *folds*, y en cada iteración (80% entrenamiento, 20% validación) se ajustaron tanto el escalado como el modelo únicamente sobre los datos de entrenamiento, evaluando después en el *fold* de validación. Esta estrategia asegura que las métricas se calculen siempre sobre datos no vistos. En cada *fold* se midió el error cuadrático medio (MSE), el error absoluto medio (MAE) y el coeficiente de correlación de Pearson (PCC).
- **Evaluación con test fijo**: donde, se separó de forma independiente un 20% de los datos como conjunto de test constante, manteniendo el 80% restante para entrenamiento. Este *split* reproducible permite comparar directamente con resultados reportados en la literatura.

Este doble esquema de validación asegura tanto la robustez estadística (mediante la validación cruzada) como la comparabilidad práctica (mediante el test fijo).

La arquitectura del modelo puede resumirse en el flujo representado en la *Figura 8*. A partir del *dataset* S2648, el proceso comienza con la descarga y parseo de estructuras PDB, de donde se extraen las secuencias correspondientes de cada mutación. Posteriormente, se genera un vector de características compuesto por tres bloques que se mencionan en el apartado de preprocesamiento.

Figura 8. Pipeline modelo XGBoost



Nota: en esta imagen podemos apreciar el funcionamiento completo del modelo desarrollado con el enfoque SAAFEC-CEQ.

Estos tres flujos se integran en un único vector en la etapa de *Feature Engineering*, que es escalado antes de alimentar las dos ramas de evaluación, la basada en validación cruzada y la otra con *split* fijo. De este modo, el esquema refleja cómo el modelo combina información heterogénea y la somete a dos procesos de evaluación complementarios, cuyas salidas confluyen en los resultados finales.

En cuanto a los parámetros de XGBoost, se emplearon configuraciones típicas para problemas de regresión en bioinformática, con un número de estimadores en el rango de 300–400, profundidad máxima de 4–5 y un *learning rate* moderado (0.05). Estos valores han conseguido asegurar un equilibrio entre la capacidad predictiva y el control del *overfitting*, obteniendo unos resultados muy cercanos a los vistos en los diferentes artículos tomados como referencia.

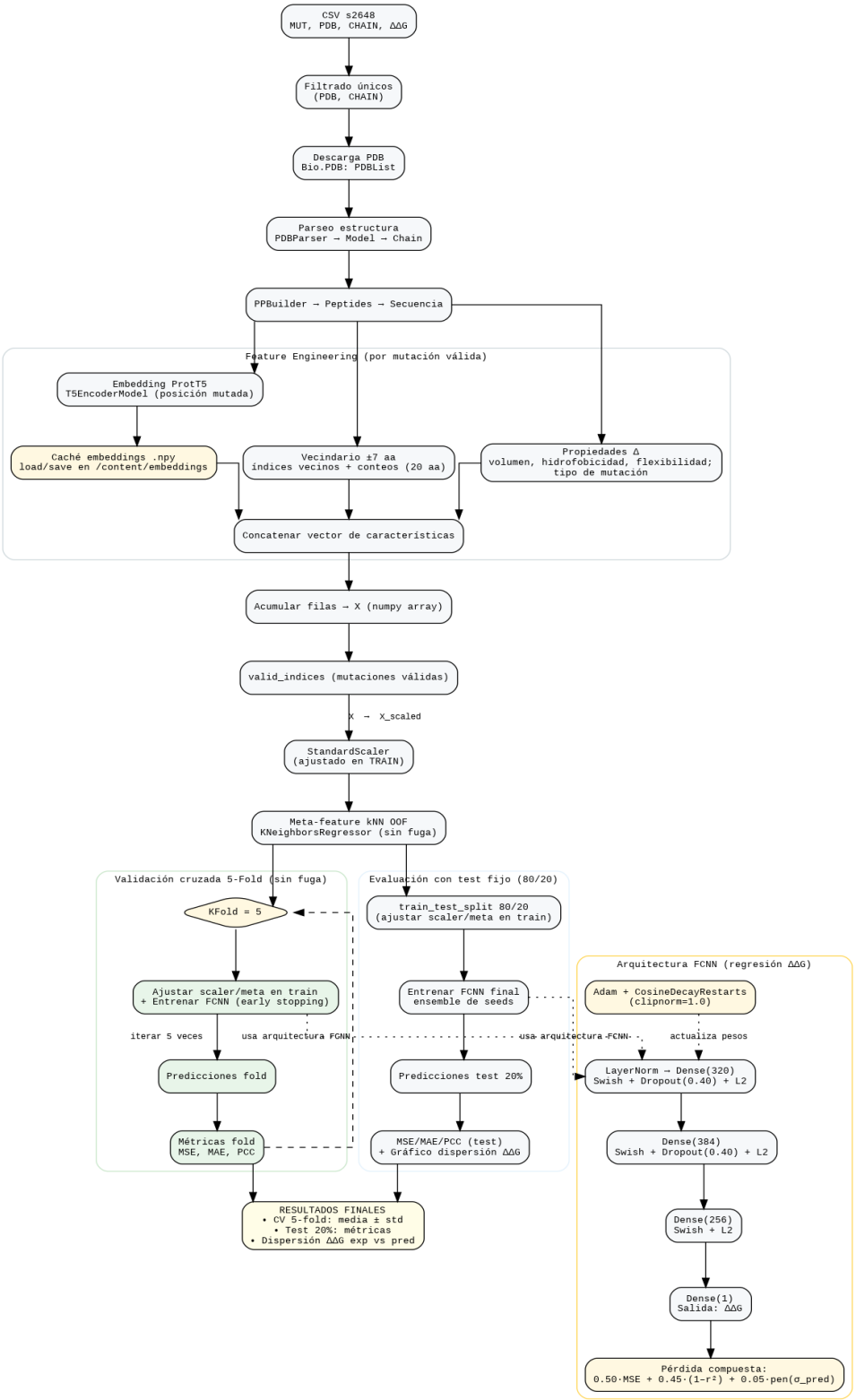
4.5. Modelo FCNN

El modelo FCNN usa el mismo preprocesamiento que el modelo XGBoost, esto permitió comparar directamente el rendimiento de un modelo de *gradient boosting* con el de NN clásica sobre el mismo espacio de características.

Como se comentó anteriormente, se probaron numerosas variaciones en el número de capas, número de neuronas y funciones de activación. Los resultados mostraron que una arquitectura relativamente de complejidad intermedia ofrecía el mejor rendimiento entre precisión, estabilidad y coste computacional, superando a arquitecturas más simples o complejas.

En la *Figura 9* se muestra un diagrama con el pipeline completo de la FCNN. Este refleja un flujo de trabajo prácticamente idéntico al utilizado en el modelo de XGBoost, lo cual permite realizar comparaciones directas entre ambos modelos. Sin embargo, es importante señalar que, a pesar de esta similitud metodológica, la red neuronal no alcanzó el mismo nivel de precisión que el modelo basado *en gradient boosting.*

Figura 9. Pipeline modelo FCNN



Nota: En esta imagen podemos ver el pipeline completo del modelo FCNN

La arquitectura interna de la FCNN y sus parámetros de entrenamiento se muestran en la Figura Y, donde, tras una capa inicial de normalización de entrada, la red se compone por tres bloques densos de tamaño intermedio:

- **Bloque 1**: 320 neuronas, activación Swish, regularización L2 y dropout del 40%.
- **Bloque 2**: 384 neuronas, también con Swish, L2 y dropout del 40%.
- **Bloque 3**: 256 neuronas con Swish y regularización L2.

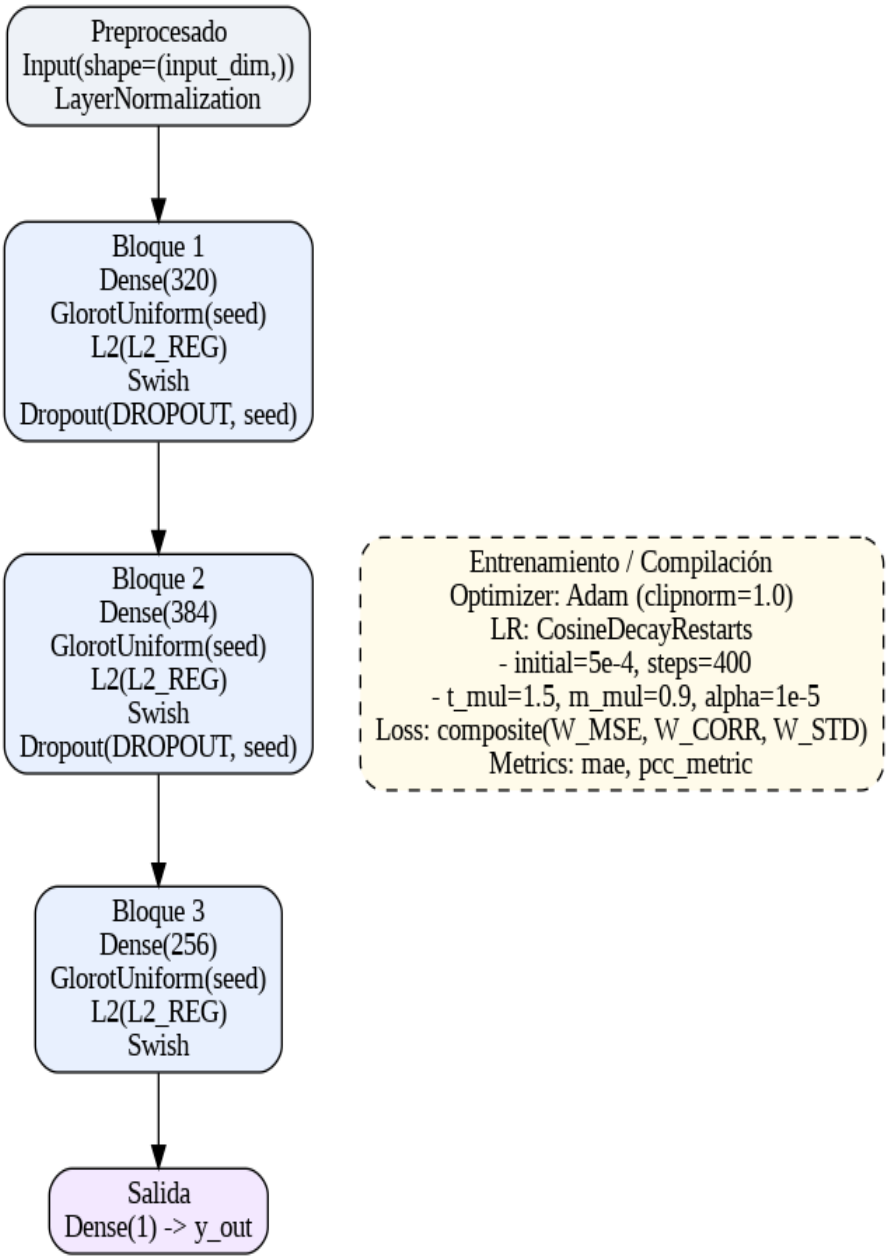
La capa de salida es una única neurona lineal, responsable de predecir el valor de ΔΔG. Esta configuración se obtuvo tras un proceso experimental iterativo en el que se evaluaron arquitecturas más profundas y simples, encontrando que este diseño equilibrado maximizaba la precisión evitando el sobreajuste y manteniendo bajo el coste computacional.

El entrenamiento incorporó la siguiente configuración:

- Optimización **Adam** y control de gradientes mediante **clipnorm**=1.0.
- Programación de la tasa de aprendizaje mediante **CosineDecayRestarts** (baja suavemente siguiendo una curva coseno y luego se reinicia en ciclos), lo que ayuda a escapar de mínimos locales y mejorar la convergencia.
- Función de pérdida compuesta, diseñada específicamente para este problema, que combina tres componentes:
    - **MSE (50%)**, para reducir el error medio cuadrático.
    - Penalización de **correlación (45%)**, que fuerza a las predicciones a mantener coherencia con la tendencia global.
    - Penalización de **desviación estándar (5%)**, para evitar colapsos en la varianza de las salidas.

Las métricas de evaluación durante el entrenamiento fueron las mencionadas anteriormente: MAE, MSE y PCC.

Figura 10. Arquitectura FCNN y parámetros de entrenamiento



Nota: En esta imagen se muestra la arquitectura por capas de la FCNN junto con los parámetros utilizados para el entrenamiento/compilación de dicha red.

4.6. Modelo cuántico análogo a XGBoost

La parte inicial del pipeline del modelo QNN cambia un poco respecto a las anteriores ya que parte de un vector de características derivado de:

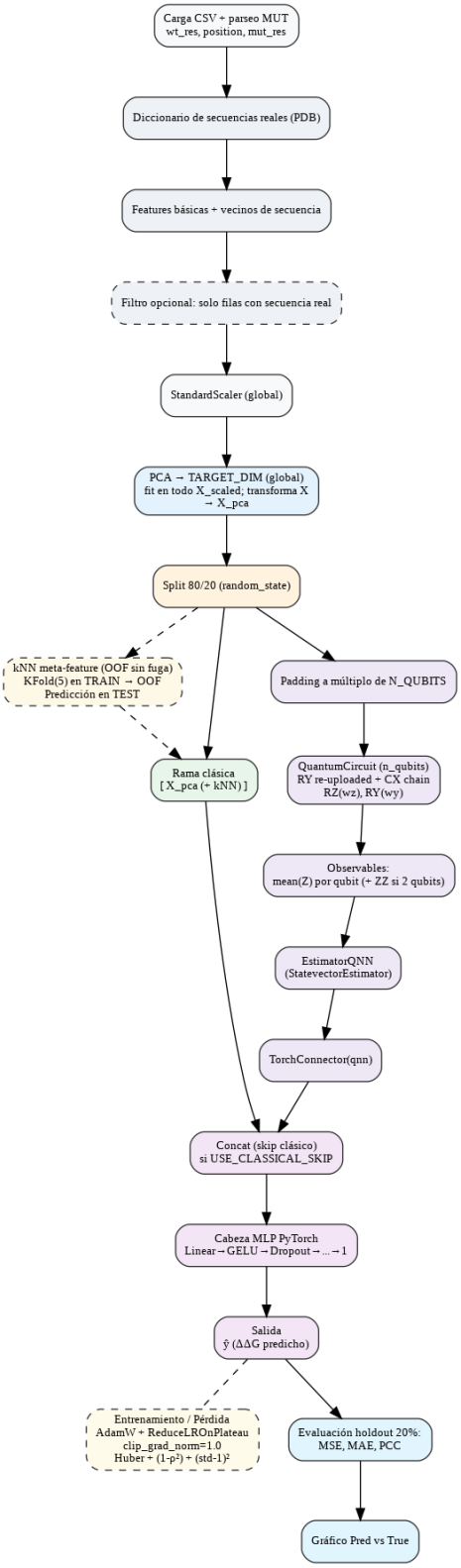
- Propiedades fisicoquímicas básicas
- Información de vecinos en la secuencia
- Codificación *one-hot* de los residuos implicados

En este caso no usamos *embeddings* ProtT5 sino que codificamos con *one-hot* los residuos implicados (*wild-type* y mutado). Tras esto, los vectores se escalan y se reducen mediante PCA a un espacio de 24 dimensiones, el cual se ajusta a los 4 qubits disponibles en el circuito.

La fase de experimentación siguió un enfoque incremental y exploratorio. Se ensayaron diferentes configuraciones de circuitos cuánticos (*ansatz*), distintos números de qubits, técnicas de *reuploading* de parámetros y varios tipos de estimadores (como StatevectorEstimator frente a métodos con *shots*). También se probaron distintas formas de integración con ramas clásicas adicionales. Este proceso requirió un esfuerzo considerable de desarrollo y ajuste y constituyó la parte más compleja de implementar dentro del marco del TFM.

En la *Figura 11* se muestra el pipeline completo del modelo QNN, donde se observa el flujo desde la construcción de secuencias reales hasta la evaluación final. Este pipeline incluye la posibilidad de añadir un *meta-feature* derivado de un modelo kNN en modo *out-of-fold* (OOF), el cual enriquece la rama clásica del modelo sin introducir fuga de información.

Figura 11. Pipeline modelo QNN



Nota: En esta imagen se muestra el pipeline completo de la QNN

La arquitectura final seleccionada para el modelo QNN se muestra en la Figura 12, donde se combina un bloque cuántico con una rama clásica integrada en un perceptrón multicapa (MLP). Esta organización permite aprovechar la capacidad de los circuitos cuánticos para generar representaciones no lineales y, al mismo tiempo, complementar dicha información con un procesamiento clásico adicional. A continuación, entraremos en detalle en la arquitectura.

1. **Entrada cuántica (X_pca padded)**

   El vector de entrada proviene del preprocesamiento descrito anteriormente, aunque escalado y reducido mediante PCA. Dado que el número de qubits debe ser un múltiplo fijo, el vector resultante se rellena (*padding*) hasta ajustarse a los 4 qubits disponibles.

2. **Circuito cuántico parametrizado**

   La información se introduce en los qubits mediante rotaciones RY(x) con estrategia de *re-uploading*, lo que permite representar secuencialmente todas las características aun utilizando un número limitado de qubits. Posteriormente, se introducen puertas de entrelazamiento CX, que generan dependencias entre qubits y enriquecen la capacidad expresiva del circuito. Finalmente, cada qubit incluye rotaciones adicionales RZ y RY con parámetros libres (wz, wy), los cuales se ajustan durante el entrenamiento y cumplen el papel de pesos cuánticos dentro de la red.

3. **Observables y función de medida**

   La salida del circuito se obtiene midiendo operadores en la base de Pauli, que es el tipo de observador de nuestra QNN. Normalmente, cada qubit contribuye con un observable tipo Z, promediando su expectativa (mean(Z)). Además, en el caso particular de emplear 2 qubits, el circuito incorpora un observable adicional ZZ, que captura la correlación conjunta entre ambos qubits. Para configuraciones con un

número mayor de qubits (N > 2), se utiliza únicamente el observable medio de Pauli-Z en cada qubit. Esta decisión evita la necesidad de incluir todos los posibles términos de correlación multiqubit (ZZ, ZZZ, …), cuyo número crece de forma exponencial con N y que incrementarían significativamente la complejidad y el coste computacional del entrenamiento. Estos valores medidos constituyen la representación cuántica que alimentará al modelo híbrido.

4. **Integración con la rama clásica**

La salida cuántica obtenida a partir de los observables se concatena con las características clásicas, previamente procesadas mediante PCA (y, de forma opcional, enriquecidas con un *meta-feature* generado mediante kNN). Este paso de integración permite combinar la capacidad no lineal del circuito cuántico con la robustez de las representaciones clásicas, configurando un modelo híbrido.

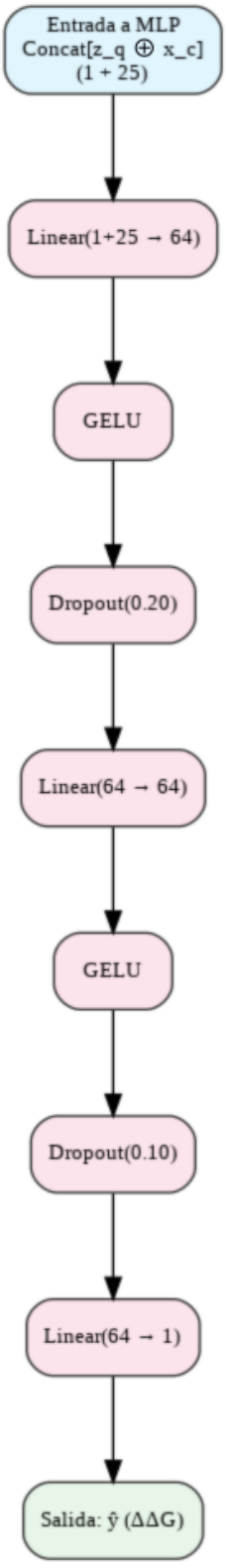
5. **Bloque MLP clásico final**

Sobre la representación híbrida $[z_q \oplus x_c]$, se incorpora una cabeza clásica formada por un perceptrón multicapa (MLP) en PyTorch. Este MLP incluye:

- Una primera capa densa de 64 neuronas + activación GELU + *dropout* (p=0.20).
- Una segunda capa densa de 64 neuronas + GELU + *dropout* (p=0.10).
- Una capa de salida lineal (64→1) que devuelve la predicción de ΔΔG.

En la Figura 13 podemos ver podemos ver con más detalle la arquitectura de esta MLP.

Figura 13. Arquitectura cabeza MLP



Nota: En esta imagen podemos observar la arquitectura de la MLP

6. Entrenamiento y función de pérdida

El modelo se entrena de forma *end-to-end* (tanto la parte clásica como híbrida) utilizando el optimizador **AdamW** con reducción adaptativa de la tasa de aprendizaje (**ReduceLROnPlateau**). La función objetivo es una pérdida compuesta que combina tres términos:

- **Huber Loss** (55%), que penaliza los errores de predicción con robustez frente a valores atípicos.
- **$1-\rho^2$ (correlación)** (40%), que fomenta la maximización del coeficiente de correlación de Pearson entre valores predichos y experimentales.
- **Penalización de varianza** (5%), que asegura que la dispersión de las predicciones se mantenga cercana a la de los valores reales (en z-score).

La evaluación se llevó a cabo con un *split* fijo de entrenamiento/prueba (80/20) y obteniendo las mismas métricas de error (MSE, MAE) y correlación (PCC) que en los otros modelos.

Capítulo 5: Presentación de análisis, resultados y discusión

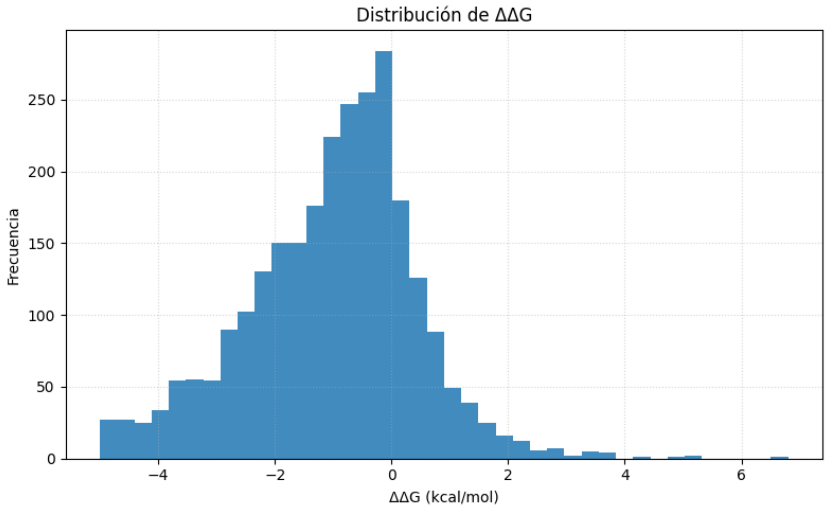
Este capítulo presenta los resultados experimentales de los tres modelos desarrollados (XGBoost, FCNN y QNN), describe el protocolo de evaluación y los estadísticos utilizados, reúne las tablas comparativas, muestra las curvas de entrenamiento y las gráficas de rendimiento, y cierra con una discusión crítica de hallazgos y limitaciones.

1. Análisis descriptivo de la muestra

El conjunto de datos utilizado, S2648, recoge un total de 2.648 mutaciones puntuales en proteínas junto con sus valores experimentales de variación de energía libre de plegamiento (ΔΔG). Antes de aplicar los modelos, se realizó un análisis exploratorio de datos (EDA) con el fin de comprender mejor la distribución de las variables y las características generales de la muestra.

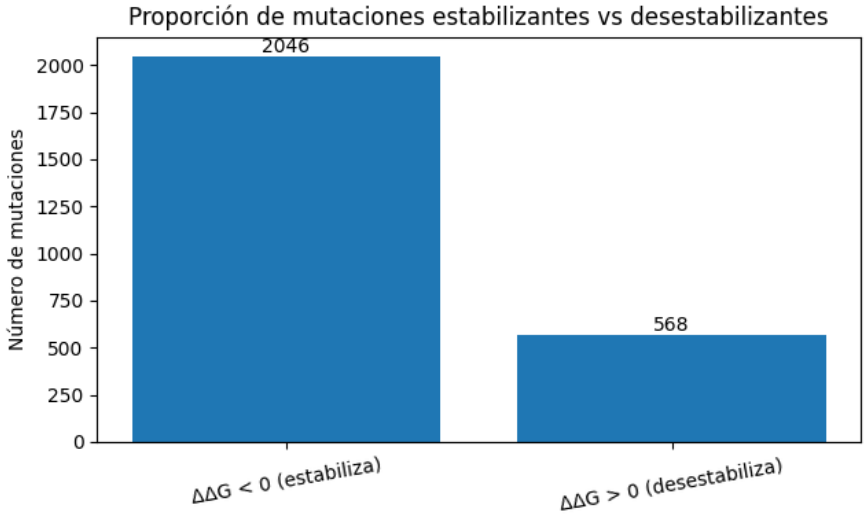
En la *Figura 14* se muestra la distribución de ΔΔG, que presenta una clara asimetría con una mayor concentración de valores negativos, lo cual indica que la mayoría de las mutaciones tienden a ser estabilizantes. Esta tendencia también se refleja en la *Figura 15*, donde se observa que aproximadamente un 80% de las mutaciones son estabilizantes ($\Delta\Delta G < 0$) mientras que alrededor del 20% resultan desestabilizantes ($\Delta\Delta G > 0$). El diagrama *boxplot, Figura 16,* confirma además la presencia de valores atípicos, especialmente en el rango positivo, lo que puede significar que algunas mutaciones desestabilizantes producen efectos extremos sobre la estabilidad proteica.

Figura 14. Distribución de DDG



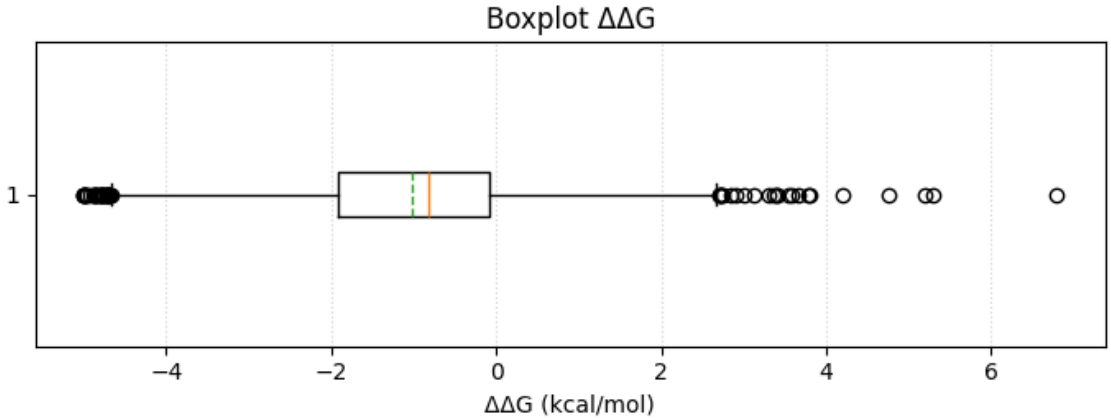
Nota: En esta imagen se muestra la distribución de los valores de DDG de las proteínas del dataset.

Figura 15. Proporción de mutaciones estabilizantes/desestabilizantes



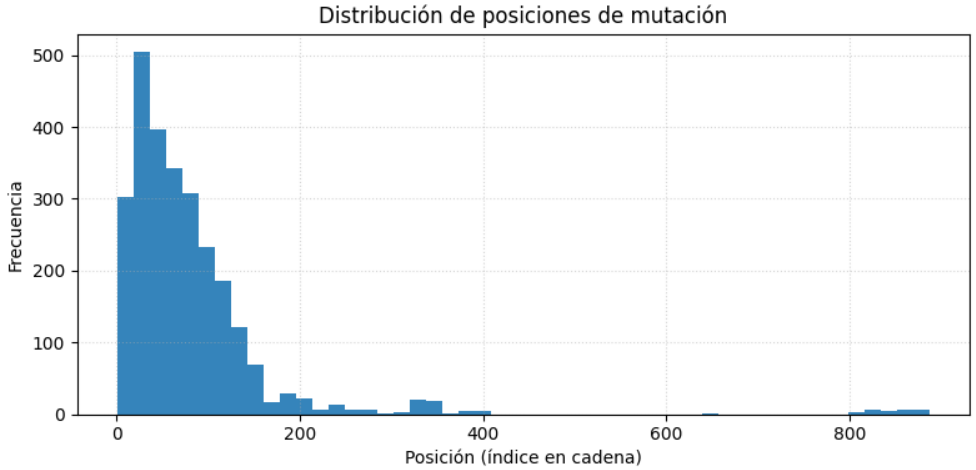
Nota: en esta imagen podemos ver la distribución de los tipos de mutaciones según si son estabilizantes o desestabilizantes y podemos observar como la clase estabilizante es 4 veces más común.

Figura 16. Boxplot DDG



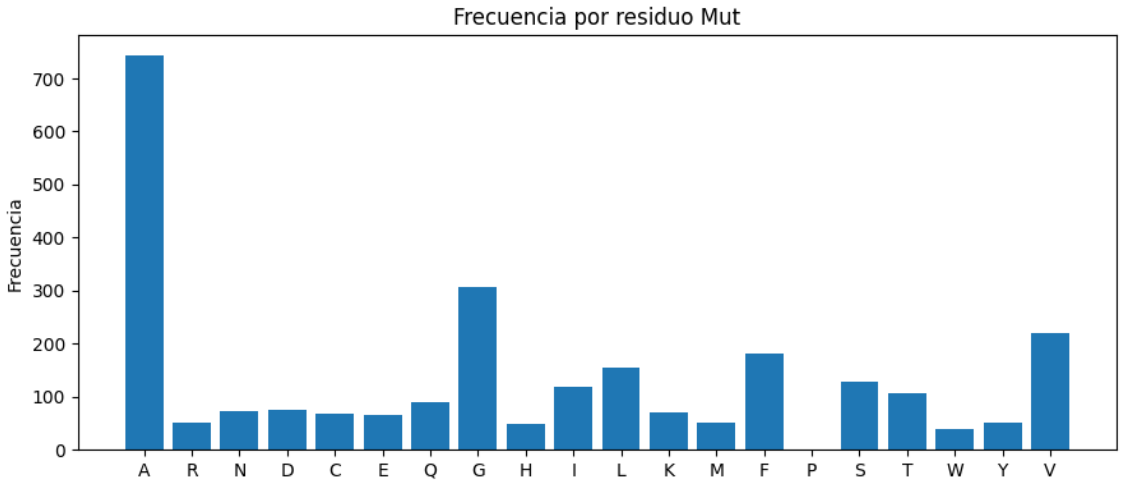
En cuanto a las posiciones de mutación dentro de las cadenas (*Figura 18*), se observa que estas se concentran mayoritariamente en las regiones iniciales de las proteínas, aunque existen ejemplos dispersos a lo largo de toda la secuencia. A nivel de aminoácidos, la *Figura 19* muestra la frecuencia de aparición de residuos originales (WT) y mutados (Mut), donde destacan especialmente las mutaciones hacia alanina (A), reflejando su uso frecuente en experimentos de mutagénesis dirigida.

Figura 18. Distribución de posición de mutación



Nota: en la distribución de posiciones podemos ver como la gran mayoría de estas se agrupa al principio de la cadena de aminoácidos

Figura 19. Frecuencia de aparición de aminoácido como mutación



Nota: podemos ver como la mayoría de las mutaciones tienden hacia alanina

Este análisis exploratorio confirma que la muestra no es completamente balanceada ni en términos de valores ΔΔG ni en la distribución de residuos, lo cual es relevante para la posterior interpretación de los resultados de los modelos.

2. Estadísticos y protocolos de evaluación

Como hemos comentado anteriormente, los estadísticos empleados para evaluar el rendimiento de los modelos han sido el coeficiente de correlación de Pearson (PCC), el error cuadrático medio (MSE) y el error absoluto medio (MAE). La elección de estas métricas se debe tanto a su uso recurrente en los diversos artículos sobre predicción de estabilidad proteica como a que permiten disponer de una visión complementaria del rendimiento. Mientras PCC mide la capacidad del modelo de capturar tendencias globales, MSE y MAE cuantifican la magnitud de los errores.

En cuanto a los protocolos de evaluación, se adoptaron dos enfoques complementarios. Por un lado, la validación cruzada 5-fold, que aporta robustez estadística al evaluar el modelo en diferentes particiones del conjunto de datos y permite reducir la dependencia de un único Split. Por el

otro lado, se empleó un conjunto de prueba fijo (80/20), lo que asegura comparabilidad práctica con trabajos previos que utilizan esta misma estrategia y ofrece un punto de referencia estable sobre el que contrastar resultados.

3. Resultados y discusión

En este apartado se van a presentar los resultados obtenidos de los tres modelos implementados (XGBoost, FCNN y QNN), considerando tanto la validación cruzada como la evaluación en un conjunto de prueba fijo. Para facilitar la interpretación, los resultados se muestran en forma de tablas con métricas agregadas y gráficas que ilustran tanto el proceso de entrenamiento como la relación entre valores predichos y experimentales.

1. Tablas de resultados principales obtenidos

En las *Tablas 1, 2 y 3* se presentan los resultados obtenidos por cada modelo (XGBoost, red neuronal clásica y QNN), tanto en validación cruzada 5-fold como en la evaluación sobre el conjunto de test fijo (80/20). Se reportan las métricas PCC, MSE y MAE, que permiten evaluar desde distintos ángulos la calidad de las predicciones.

Tabla 1. Resultados modelo XGBoost

Fold	PCC	MSE	MAE
1	0.688	1.115	0.812
2	0.700	1.139	0.814
3	0.668	1.284	0.849
4	0.584	1.358	0.900
5	0.658	1.205	0.823
Media ± DE	0.660 ± 0.041	1.220 ± 0.090	0.840 ± 0.033

Evaluación test fijo (80/20)	PCC	MSE	MAE
	0.703	1.070	0.783

El modelo XGBoost mostró un rendimiento global superior, alcanzando en el test fijo un PCC de 0.703, lo que refleja una buena capacidad para capturar las tendencias de los valores experimentales. Además, obtuvo el menor error absoluto (MAE=0.783) y cuadrático (MSE=1.070) entre los tres enfoques, consolidándose como una solución robusta y consistente. Su validación cruzada también presentó desviaciones estándar relativamente bajas, lo que sugiere una mayor estabilidad frente a diferentes particiones de los datos.

Tabla 2. Resultados modelo red neuronal clásica

Fold	PCC	MSE	MAE
1	0.601	1.375	0.812
2	0.660	1.265	0.814
3	0.624	1.440	0.849
4	0.590	1.402	0.900
5	0.595	1.377	0.823
Media ± DE	0.614 ± 0.026	1.372 ± 0.058	0.880 ± 0.019

Evaluación test fijo (80/20)	PCC	MSE	MAE
	0.596	1.387	0.895

La FCNN presentó un desempeño intermedio, con un PCC de 0.614 ± 0.026 en el *5-fold* y un valor de 0.596 en el test fijo. Aunque sus errores fueron algo mayores (MSE=1.387, MAE=0.895), se aprecia que el modelo logra mantener una correlación razonable con los valores experimentales, si bien con mayor variabilidad y cierta tendencia al sobreajuste.

Tabla 3. Resultado modelo QNN

Fold	PCC	MSE	MAE
1	0.567	1.339	0.927
2	0.515	1.443	0.981
3	0.552	1.500	1.018
4	0.534	1.545	0.957
5	0.521	1.321	0.972
Media ± DE	0.533 ± 0.022	1.430 ± 0.088	0.970 ± 0.030

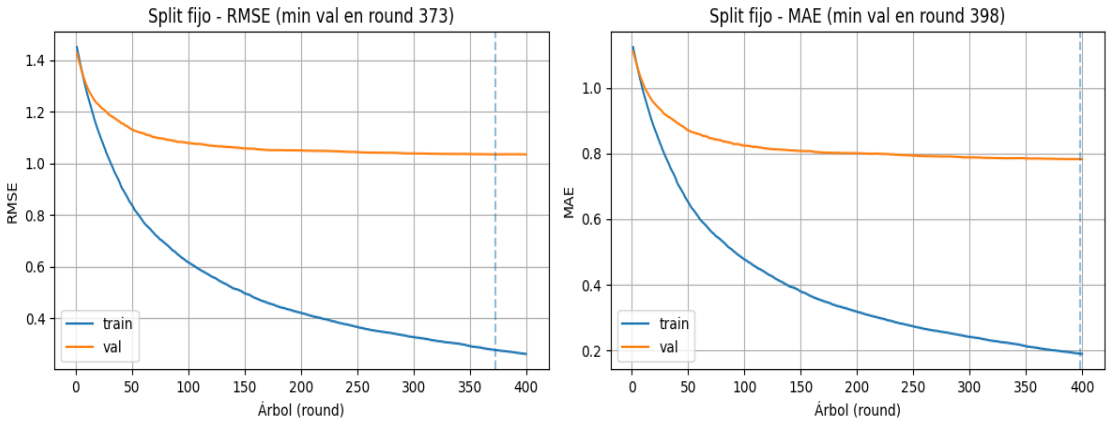
Evaluación test fijo (80/20)	PCC	MSE	MAE
	0.540	1.361	0.893

Por último, el modelo QNN ofreció resultados más modestos en términos de correlación (PCC≈0.533 en validación y 0.54 en el test fijo) y con errores mayores (MSE≈1.430 en validación). No obstante, es relevante destacar que la QNN es un enfoque exploratorio y aún emergente, sujeto a limitaciones de hardware y software.

2. Curvas de entrenamiento y gráficas comparativa

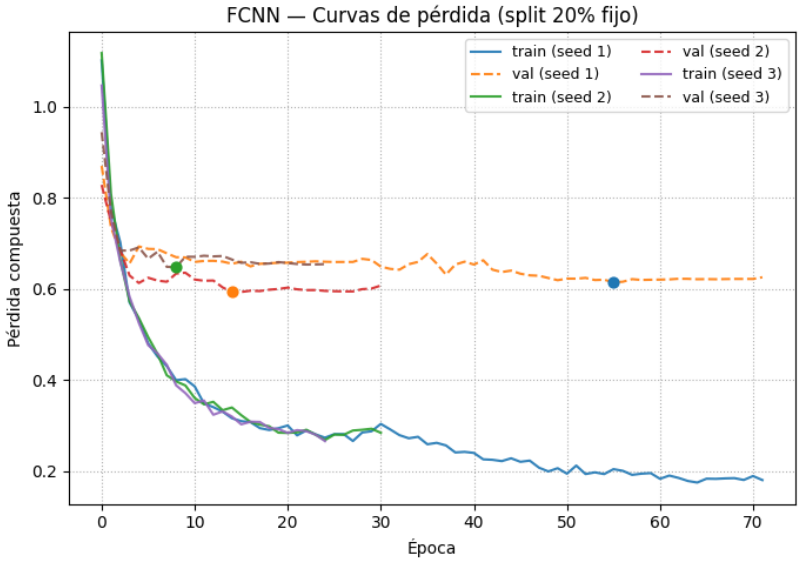
Con el objetivo de analizar el comportamiento de los modelos durante el entrenamiento, se presentan las curvas de pérdida correspondientes a cada uno de ellos. Estas gráficas permiten evaluar cómo evolucionan las métricas de error tanto en el conjunto de entrenamiento como en el de validación, identificando la estabilidad del aprendizaje y posibles casos de sobreajuste.

Figura 20. Pérdida entrenamiento XGBoost split (80/20)



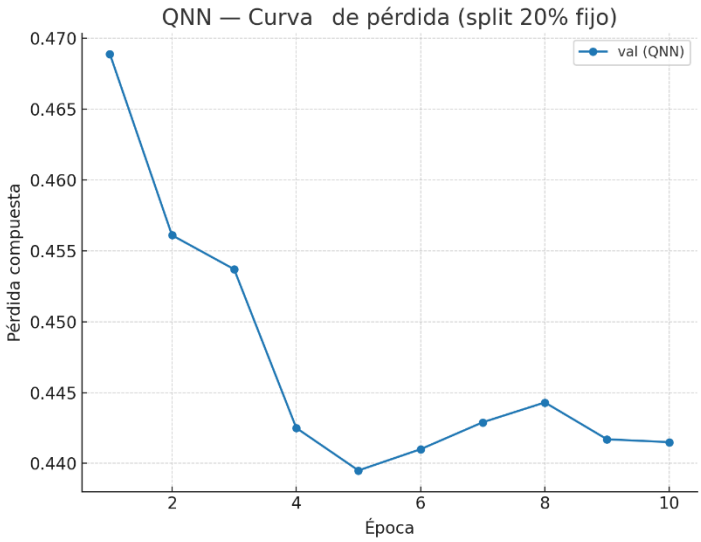
En el caso de XGBoost (*Figura 20*), se observa una convergencia progresiva y estable de las métricas RMSE y MAE a lo largo de los *boosting* rounds. El error de validación tiende a estabilizarse alrededor de la iteración 70, indicando un buen ajuste sin señales evidentes de *overfitting*.

Figura 21. Pérdida entrenamiento FCNN split (80/20) 3 runs



La FCNN (*Figura 21*) muestra unas curvas de pérdida más variables, especialmente entre las distintas semillas de inicialización, lo que refleja la sensibilidad de la red a la configuración de partida. Sin embargo, gracias al uso de regularización (*dropout*, L2 y reducción adaptativa de la tasa de aprendizaje), el entrenamiento logra estabilizarse tras unas 20–30 épocas, alcanzando un punto en el que la pérdida de validación se mantiene en una zona estable sin degradación marcada.

Figura 22. Perdida entrenamiento QNN split (80/20)

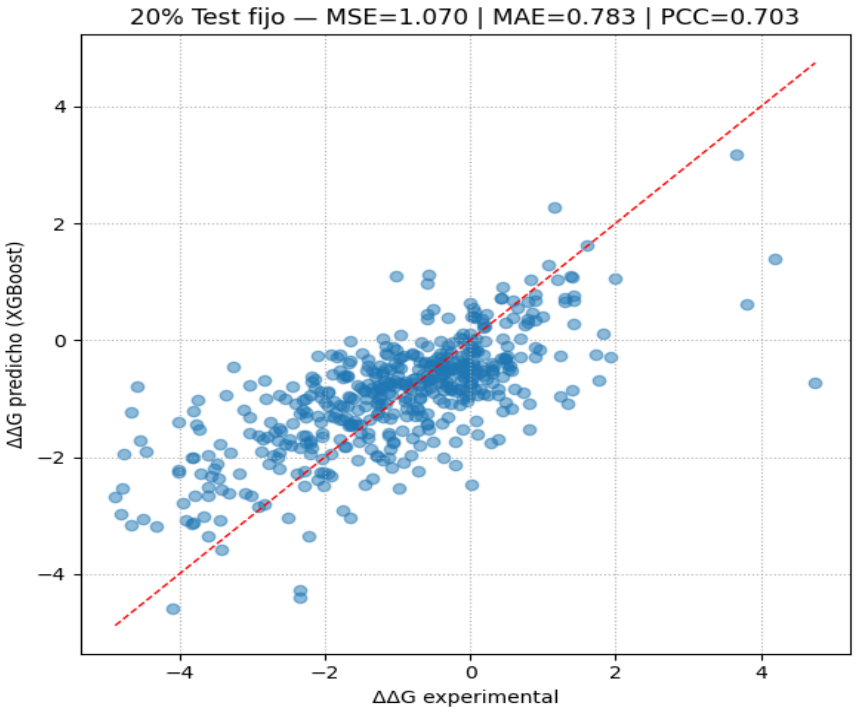


Por último, la QNN (*Figura 22*) presenta una evolución con mayor fluctuación en la pérdida de validación. A pesar de ello, se observa una tendencia descendente en las primeras épocas y cierta estabilización posterior, lo cual sugiere que el modelo híbrido es capaz de aprender representaciones útiles, aunque con limitaciones en comparación con los enfoques clásicos.

3. Gráficos de dispersión Predicción vs Real

A continuación, se presentan los gráficos de dispersión correspondientes a las predicciones frente a los valores experimentales en el conjunto de prueba fijo (20%), para los tres modelos implementados: XGBoost, FCNN y QNN. La línea diagonal roja representa el ajuste perfecto (predicción idéntica al valor experimental), lo que permite evaluar visualmente la capacidad de los modelos para aproximarse a los datos reales.

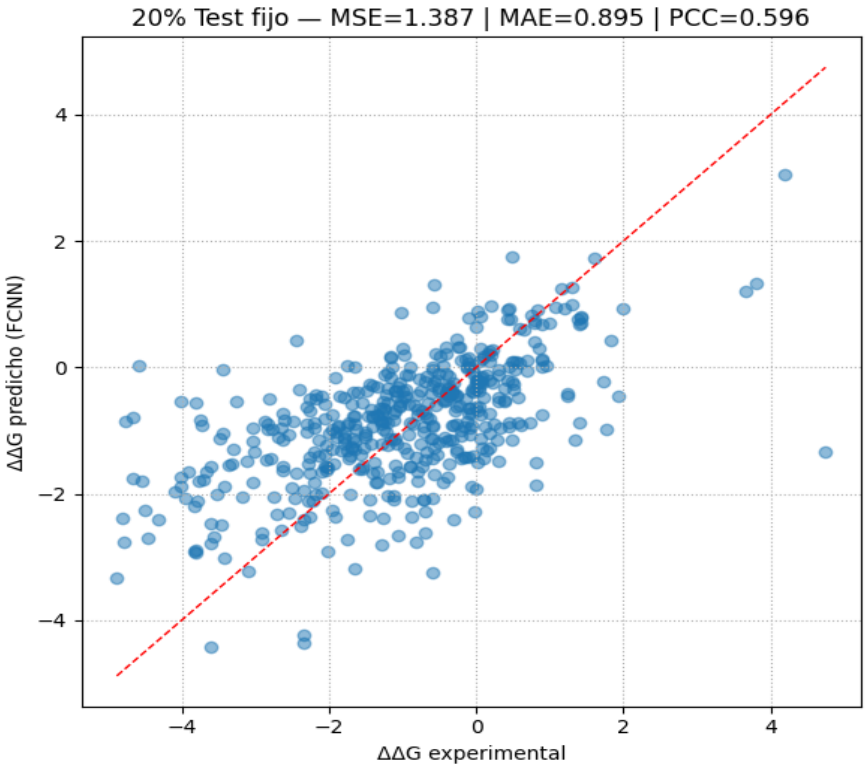
Figura 23. Gráfica de dispersion Pred vs True XGBoost



En el caso de XGBoost (*Figura 23*), se observa una nube de puntos más alineada con la diagonal, lo que concuerda con el mayor valor de correlación

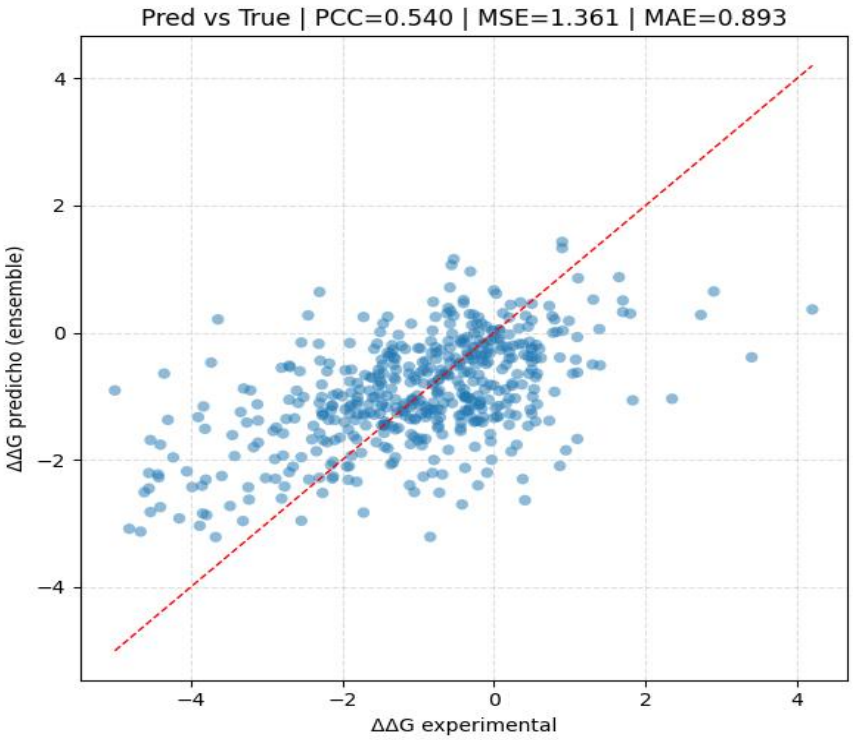
(PCC = 0.703) y con los errores más bajos (MSE = 1.070, MAE = 0.783). Esto indica que el modelo consigue capturar adecuadamente las tendencias generales y mantener una buena precisión numérica en sus predicciones.

Figura 24. Gráfica de dispersión Pred vs True FCNN



Por su parte, la FCNN (*Figura 24*) muestra una dispersión más amplia alrededor de la diagonal, con menor correlación (PCC = 0.596) y errores más altos (MSE = 1.387, MAE = 0.895). Aunque consigue reflejar parcialmente la tendencia global, la mayor variabilidad sugiere un ajuste menos consistente en comparación con XGBoost.

Figura 25. Gráfica de dispersión Pred vs True QNN



Finalmente, la QNN (*Figura 25*) presenta un comportamiento más modesto, con menor correlación (PCC = 0.540) y errores intermedios (MSE = 1.361, MAE = 0.893). La dispersión de los puntos indica que el modelo aún no logra capturar de forma robusta la relación entre predicciones y valores reales. No obstante, estos resultados son relevantes en tanto que muestran la viabilidad de un enfoque híbrido cuántico-clásico en un problema realista, a pesar de las limitaciones.

4. Comparación de los modelos implementados

A continuación, se muestra la *Tabla 4,* donde se comparan los resultados obtenidos por los tres modelos con los distintos procesos de evaluación para realizar una breve comparación posterior a esta.

Tabla 4. Comparación de resultados obtenidos en los modelos implementados

Modelo	Proceso de evaluación	PCC	MSE	MAE
XGBoost	Evaluación test fijo (80/20)	0.703	1.070	0.783
	Media ± DE	0.660 ± 0.041	1.220 ± 0.090	0.840 ± 0.033
FCNN	Evaluación test fijo (80/20)	0.596	1.387	0.895
	Media ± DE	0.614 ± 0.026	1.372 ± 0.058	0.880 ± 0.019
QNN	Evaluación test fijo (80/20)	0.540	1.361	0.893
	Media ± DE	0.533 ± 0.022	1.430 ± 0.088	0.970 ± 0.030

Los resultados muestran que XGBoost alcanza el mejor rendimiento global, con un PCC de 0.703 y los errores más bajos (MSE=1.070, MAE=0.783) en el conjunto de prueba fijo. Estos valores son consistentes con la validación cruzada (PCC≈0.66 ± 0.04), que mostró baja variabilidad entre *folds*, lo que refuerza su estabilidad y capacidad de generalización frente a distintas particiones de los datos.

La FCNN presentó un desempeño intermedio, con un PCC de 0.596 y errores algo mayores (MSE=1.387, MAE=0.895) en el test fijo. Aunque su rendimiento fue inferior al de XGBoost, la validación cruzada (PCC≈0.61 ± 0.03) confirma que el modelo mantiene correlaciones consistentes, pese a mostrar mayor sensibilidad a la inicialización y cierta variabilidad entre ejecuciones.

Por su parte, la QNN obtuvo un PCC más modesto (≈0.540 en test fijo y ≈0.533 ± 0.022 en validación cruzada), quedando por detrás en correlación respecto a los otros dos modelos. No obstante, resulta relevante destacar que, aunque inferior en correlación, la QNN logró superar a la FCNN en errores absolutos y cuadráticos dentro del test fijo (MSE=1.361 < 1.387 y MAE=0.893 < 0.895).

En conclusión, la comparación entre los tres enfoques refleja un claro liderazgo de XGBoost en estabilidad y precisión, mientras que la FCNN ofrece un rendimiento intermedio con potencial de mejora mediante optimización adicional o más datos. La QNN, pese a sus limitaciones, demuestra que los modelos híbridos cuántico-clásicos pueden ya alcanzar métricas comparables a las de arquitecturas neuronales clásicas, lo que refuerza la relevancia de seguir explorando esta línea de investigación de cara al futuro.

4. Comparación con otros modelos de la literatura

Para contextualizar los resultados obtenidos con los tres modelos implementados en este trabajo, en la *Tabla 4* se comparan con distintos métodos de predicción de ΔΔG reportados en la literatura y evaluados sobre el mismo conjunto de datos S2648. En esta tabla se muestran métricas de correlación (PCC/) y error (MSE) de una selección representativa de modelos clásicos y recientes.

Tabla 4. Comparación de modelos de la literatura con los modelos implementados

Método	PCC	MSE
XGBoost (SAAFEC-CEQ)	0.703	1.070
mCSM	0.690	1.140
PoPMuSiC	0.610	1.370
ProS_GNN	0.608	1.207
FCNN	0.596	1.387
GeoPPI	0.584	1.188
INPS3D	0.580	1.440
QNN	0.540	1.361
MutaBind2	0.532	1.228
SDM2	0.480	2.130
TopGBT	0.451	1.352

Nota: En establa se muestran los modelos ordenados según su PCC, aunque como podemos ver en algunos casos el MSE puede ser mayor o menor independientemente de la correlación

En esta tabla podemos observar que herramientas consolidadas como mCSM (PCC=0.690, MSE=1.140) o PoPMuSiC (PCC=0.610, MSE=1.370) alcanzan todavía un rendimiento competitivo, mientras que enfoques más recientes como GeoPPI y ProS_GNN mantienen correlaciones en torno a 0.58–0.61, con errores similares a los obtenidos en los modelos clásicos de este trabajo. Por otro lado, algunos métodos como SDM2 (PCC=0.480, MSE=2.130) o TopGBT (PCC=0.451, MSE=1.352) presentan limitaciones notables, lo que evidencia la dificultad del problema incluso para algoritmos basados en información estructural. Respecto a los métodos implementados, podemos observar que el modelo XGBoost destaca por encima de los demás en ambas métricas, mientras que el modelo FCNN se sitúa entre los 5 mejores y el modelo de QNN supera algunos de los modelos con peor rendimiento.

5. Limitaciones

El desarrollo del presente trabajo ha estado condicionado por una serie de limitaciones que es importante señalar para contextualizar adecuadamente los resultados obtenidos. En primer lugar, la falta de un conocimiento profundo previo en campos tan complejos y en constante evolución como lo son la biotecnología y la computación cuántica supuso un gran desafío inicialmente, ya que requirió un esfuerzo considerable de aprendizaje para integrar ambos ámbitos de forma coherente en este proyecto.

Por otra parte, la propia naturaleza de la computación cuántica añade restricciones significativas. Al tratarse de un área emergente, aún no existe una estandarización clara ni consistencia plena en los resultados de los diferentes *frameworks* y bibliotecas. A ello se suma la imposibilidad de acceder a hardware cuántico real, lo que obligó a trabajar exclusivamente con simuladores. Estos simuladores, si bien han resultado esenciales para el desarrollo de la QNN, estos se ven muy afectados por la exponencialidad de los tiempos de ejecución debido a pequeñas variaciones en parámetros como el número de qubits o la profundidad del circuito, lo que implicaba incrementos desproporcionados en el coste computacional.

En la práctica, la ejecución de los experimentos también estuvo limitada por las restricciones de hardware y la limitación de tiempo en las sesiones de Google Colab, lo que supuso la necesidad de reducir en ocasiones el número de repeticiones o ajustes de hiperparámetros para hacer viable la obtención de resultados.

En general, estas limitaciones no desmerecen los avances logrados, pero sí ponen de relieve que los resultados deben interpretarse en el marco de un trabajo exploratorio, con margen de mejora tanto en términos metodológicos como computacionales a medida que la tecnología cuántica y los recursos disponibles continúen evolucionando.

Capítulo 6: Conclusiones

1. Resumen del proyecto y conclusiones

En este proyecto se abordó de manera integral el problema de la predicción de ΔΔG en mutaciones puntuales de proteínas, cubriendo tanto los fundamentos teóricos como la implementación práctica. Para ello se recurrió a tecnologías de vanguardia, como la inteligencia artificial y la computación cuántica, con el objetivo de explorar su potencial en un contexto biotecnológico. Inicialmente, se definió el objetivo del trabajo y se introdujeron los conceptos clave de los distintos campos implicados, con el fin de facilitar la comprensión del problema y de las posibles soluciones a aplicar.

Posteriormente, se desarrollaron diversas aproximaciones con distintos enfoques metodológicos, evaluando su eficacia sobre un mismo conjunto de datos. Los resultados obtenidos fueron comparados tanto entre sí como frente a modelos de referencia descritos en la literatura, lo que permitió situar el estado actual del problema, identificar el grado de viabilidad de las soluciones propuestas y establecer una visión más clara sobre las fortalezas y limitaciones de cada enfoque.

En cuanto a los resultados, como se detalló en el capítulo anterior, XGBoost fue el modelo más competitivo, confirmando las expectativas y alcanzando el mejor equilibrio entre precisión y estabilidad. La FCNN obtuvo un rendimiento correcto, comparable al de modelos reportados en la literatura, aunque con mayor variabilidad y errores algo superiores. Finalmente, la QNN, pese a quedar por detrás de los otros dos enfoques desarrollados, logró métricas que superan a algunos de los otros métodos, lo que resulta especialmente relevante dado el carácter exploratorio y emergente de esta tecnología.

En conclusión, este trabajo demuestra que, aunque la computación cuántica todavía se encuentra en una fase temprana de desarrollo, su aplicación en biotecnología presenta un gran potencial. Los resultados de la QNN

evidencian que ya es posible alcanzar un rendimiento cercano al de modelos establecidos, lo que abre la puerta a futuros avances a medida que mejoren el hardware y los algoritmos cuánticos. De este modo, la integración de enfoques clásicos y cuánticos se perfila como una línea de investigación prometedora para el estudio y simulación de proteínas.

2. Prospectiva

Aunque este trabajo ha permitido implementar y comparar distintos enfoques clásicos y cuánticos para la predicción de ΔΔG en mutaciones puntuales de proteínas, aún quedan múltiples vías abiertas de desarrollo que podrían mejorar y ampliar sus resultados.

En primer lugar, una línea de continuidad natural sería el aumento de la muestra de datos. Trabajar con conjuntos de mayor tamaño y diversidad, incluyendo bases como ProTherm o SKEMPI v2, permitiría mejorar la capacidad de generalización de los modelos y validar su rendimiento en contextos más amplios.

En segundo lugar, se debería profundizar en el uso de embeddings de proteínas más avanzados (ProtT5, ESM-2 o AlphaFold-derived features), con el fin de enriquecer la representación de las mutaciones y capturar información estructural de mayor nivel. Integrar estos *embeddings* de forma híbrida con arquitecturas cuánticas podría dar lugar a un rendimiento superior.

Por el lado cuántico, sería interesante experimentar con ansatz más expresivos y mayor número de qubits, así como explorar técnicas de reducción de ruido y simuladores más avanzados que acerquen los experimentos a un escenario de hardware real. El acceso a computadores cuánticos físicos, aunque limitado en la actualidad, permitiría validar la aplicabilidad de las QNN más allá de los simuladores.

Por último, un aspecto clave para el futuro es la optimización de las funciones de pérdida y protocolos de entrenamiento híbridos. Diseñar pérdidas adaptadas específicamente al problema de predicción de ΔΔG y estudiar esquemas de entrenamiento multitarea o *transfer learning* podría mejorar la robustez de los resultados.

En conjunto, la continuación de este trabajo debería dirigirse a integrar datos más ricos, modelos más potentes y recursos cuánticos reales, con el objetivo de avanzar hacia una herramienta fiable y práctica para la simulación y predicción de mutaciones en proteínas.

3. Consideraciones finales

Llevar a cabo este TFM ha sido un gran desafío, principalmente porque ha requerido, ya no solo bastante tiempo, sino el tener que aprender sobre otras disciplinas como la biotecnología y la computación cuántica a base de preguntar a amigos, tutoriales y artículos. Y ya no solo el tener que aprender sobre esto, sino el intentar darle forma a todos estos conocimientos implementando un modelo que pueda rendir lo mejor que pueda. Ha sido un proceso duro pero muy enriquecedor a la vez, que esperemos que me sirva de base para proyectos relacionados en el futuro.

Respecto a las competencias adquiridas, me llevo la capacidad de poder entender un poco como funciona uno de los campos más importantes de la biotecnología, como es el trabajo con proteínas y el poder comprender un poco más de una tecnología tan innovadora, interesante y prometedora como lo es la computación cuántica.

Respecto a lo aprendido en el Máster en sí, puedo decir orgulloso que puedo entender gran parte de como funciona el campo de la IA, desde redes neuronales, al tratamiento de datos o a la implementación de diversos modelos.

Creo que este año ha sido duro por diversas razones, pero creo que los resultados han sido bastante buenos. Respecto a los docentes del Máster no

hay queja posible, todo el conjunto de profesores ha sido muy profesional y se ha centrado en el aprendizaje de los alumnos, mostrando mucho interés y disponibilidad. No podría haber pedido más.

También, me gustaría comentar que espero que los avances en biotecnología y computación cuántica sigan confluyendo en el futuro, permitiendo desarrollar herramientas más precisas y útiles para la simulación y predicción de mutaciones en proteínas, lo que podría suponer un bien mayor para la sociedad, permitiendo el desarrollo de nuevos fármacos y curas para enfermedades, trastornos o cualquier situación que pueda complicar la vida de una persona. Por mi parte, espero que este trabajo, aunque modesto, sirva como una pequeña contribución en esa dirección y como punto de partida para seguir explorando esta intersección entre ciencia, tecnología e innovación con el fin de ayudar un poco a los demás.

Bibliografía

Abadi, M. et al. (2016). TensorFlow: A system for large-scale machine learning. OSDI'16. https://arxiv.org/abs/1605.08695

Arute, F. et al. (2019). Quantum supremacy using a programmable superconducting processor. Nature, 574, 505–510. https://doi.org/10.1038/s41586-019-1666-5

Barenco, A., Bennett, C. H., Cleve, R., DiVincenzo, D. P., Margolus, N., Shor, P., … & Weinfurter, H. (1995). Elementary gates for quantum computation. Physical Review A, 52(5), 3457–3467. https://doi.org/10.1103/PhysRevA.52.3457

Biamonte, J. et al. (2017). Quantum machine learning. Nature, 549, 195–202. https://doi.org/10.1038/nature23474

Einstein, A., Podolsky, B., & Rosen, N. (1935). Can Quantum-Mechanical Description of Physical Reality Be Considered Complete? Physical Review, 47, 777–780. https://doi.org/10.1103/PhysRev.47.777

Goodfellow, I. et al. (2014). Generative adversarial nets. NeurIPS'14. https://arxiv.org/abs/1406.2661

Hochreiter, S. & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

IBM Quantum. (2024). Qiskit SDK API documentation (latest version). https://docs.quantum.ibm.com/api/qiskit/index

IBM Quantum. (2025). Qiskit Machine Learning Documentation. https://quantum.cloud.ibm.com/docs/guides

IBM Quantum. (2025). Qiskit SDK Documentation. https://quantum.cloud.ibm.com/docs/guides

Jumper, J., Evans, R., Pritzel, A. et al. (2021). Highly accurate protein structure prediction with AlphaFold. Nature, 596, 583–589. https://doi.org/10.1038/s41586-021-03819-2

Kennedy, J. & Eberhart, R. (1995). Particle swarm optimization. Proc. IEEE International Conference on Neural Networks, 1942–1948. https://doi.org/10.1109/ICNN.1995.488968

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. Science, 220(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 436–444. https://doi.org/10.1038/nature14539

McCarthy, J., Minsky, M. L., Rochester, N., & Shannon, C. E. (1955). A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence. https://www.dartmouth.edu/~ai50/AIhistory/dartmouth.pdf

McCarthy, J. (1960). Programs with Common Sense. In Proceedings of the Teddington Conference on the Mechanization of Thought Processes. https://jmc.stanford.edu/articles/mcc60/mcc60.pdf

Mitchell, T. M. (1997). Machine Learning. McGraw-Hill. https://www.cs.cmu.edu/~tom/mlbook.html

Preskill, J. (2018). Quantum Computing in the NISQ era and beyond. Quantum, 2: 79. https://doi.org/10.22331/q-2018-08-06-79

Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. Psychological Review, 65(6), 386–408. https://doi.org/10.1037/h0042519

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323, 533–536. https://doi.org/10.1038/323533a0

Shresth. (s.f.). It will take just 6 seconds for Google's Quantum Computer to complete 47 years of computing tasks. Medium. https://shresthj.medium.com/it-will-take-just-6-seconds-for-googles-quantum-computer-to-complete-47-years-of-computing-tasks-799b00243771

Anexos

Repositorio de GitHub donde se encuentra el notebook final con el código desarrollado:

https://github.com/marrocrod/TFM_UAX